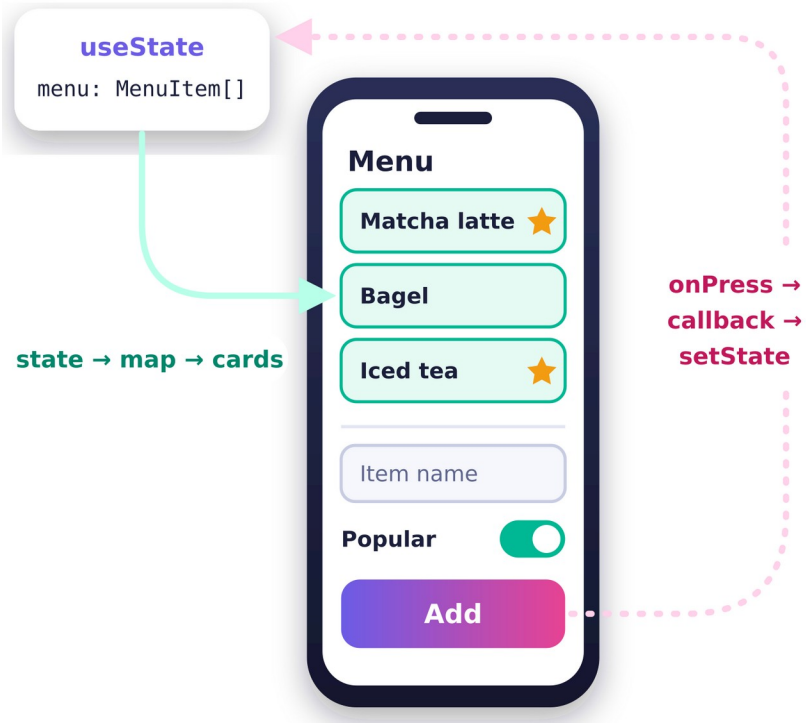
CS 147L · Coding Interview 2 Prep Kit

React Native + TypeScript · a printable study guide for the 25-minute live coding interview

Concepts · Practice with answers · Three mock interviews · Cheat sheet · Interview-day checklist



Built from course lectures 1a–5b and the official React, React Native, TypeScript, and MDN docs.

Contents

How to use this guide 3

Today's Plan 3

The 5 Golden Patterns 3

Part 1 · Concepts 7

1. Mental model 7
2. Arrow functions 8
3. map & lists 9
4. Objects & spread 12
5. useState 13
6. Components & props 15
7. Callbacks & data flow 16
8. Inputs 17
9. Conditional rendering 19
10. TypeScript 21
11. Layout 24
12. Effects 26
13. Interview craft 28

Part 2 · Practice 31

Quick drills 31

Exercises by topic 34

Build A: Café Menu Builder 37

Part 3 · Mock Interviews 39

Mock 1: Café Menu 41

Mock 2: Habit Tracker 47

Mock 3: Study Timer & Tasks 51

Rapid-fire questions with model answers 56

Part 4 · Cheat Sheet 62

Tricky stuff: wrong → right → why 62

Error decoder: message → meaning → fix 62

Golden patterns (quick reference) 63

App skeleton: type it from memory 64

Interview-day checklist 67

Answer Key 69

Drill answers 69

Exercise solutions 72

How to use this guide

This guide is the printable companion to the CS 147L Interview 2 Prep Kit website. It follows the same order as the site, so you can move between paper and screen without getting lost.

- **Short on time?** Read The 5 Golden Patterns, then the Cheat Sheet in Part 4.
- **Following the day?** Use Today's Plan below. Each sprint says what to read, what to type, and what to skip if you run long.
- **Exploring by topic?** Every topic in Part 1 opens with a one-sentence takeaway and closes with its most common trap. Practice it in Part 2 and check yourself with the rapid-fire questions in Part 3.
- **Answers stay out of sight.** Drill answers and exercise solutions are in the Answer Key at the back, so you can attempt each one first. Hints sit right under each exercise.
- **Type, do not just read.** Keep a blank Snack open at snack.expo.dev and type the code as you go.

> **The path that works best.** Get comfortable in this order: **(1)** plain JavaScript state, `map`, and arrow functions; **(2)** arrow functions used as callbacks between components; **(3)** TypeScript types added on top. Write it in JavaScript style first and let inference do most of the typing; add explicit types once it runs.

Today's Plan

Six focused sprints, about four and a half hours in total, built around your two class blocks. Interview at 4:00 pm. Times are suggestions. If a sprint runs long, skip an “If time” line rather than a break. **Spend at least half of every sprint typing in Snack, not reading.**

Time	Block and goal	Must do	If time
7:00 30 min	**Wake up and eat breakfast** Fuel first. If you feel like it, take 10 minutes to read the 5 Golden Patterns as a warm-up, nothing more.		
7:30 – 8:15 45 min	**Sprint 1: The Big Three: arrows, map, state updates** **Goal:** Make arrow functions, `map`, and immutable array updates feel automatic, because every other step builds on them.	• Read Arrow functions. The rule: with no `{` right after `=>`, the value is returned automatically; a `{` starts a function body that needs `return`. • Read map & lists, then type a one-line `map` with a `key` from memory in a blank Snack. • Read State and Spread & immutable updates. • Do the Tier 1 drills in arrows, map, and spread.	• Finish with the predict-the-output set in Drills.
8:15 – 9:00 45 min	**Sprint 2: Callbacks and inputs** **Goal:** Connect a form in a child component to state in the parent, the exact shape most of this interview is built on.	• Read Components & props: the props object and destructuring. • Read Props & callbacks: the handler lives in the parent and the child calls it. • Read Inputs: `TextInput`, `Switch`, and `Pressable`. • Do the Tier 2 exercises in callbacks and inputs. • Spend 10–15 minutes on layout: put a card’s name and label in a row with `flexDirection: "row"` and `justifyContent: "space-between"`, and give the `TextInput` `borderWidth: 1` so you can see it.	• Try the layout practice or components practice.
9:00 10 min	**Break** Stand up, stretch, refill your water. Stepping away helps the patterns settle.		

Time	Block and goal	Must do	If time
9:10 – 10:15 65 min	**Sprint 3: Full timed mock #1** **Goal:** Rehearse the real format once, end to end, so nothing about 25 minutes in Expo Snack feels new.	• Open Mock 1, start the 25-minute timer, and think aloud as you work. • Spend 15 minutes on a self-review with the rubric under the mock. Be kind and specific. • Pick your weakest step and rework it from a blank file, using the matching practice topic.	• Jot down two or three things you want on your handwritten page.
10:30 – 11:30 60 min	**Class**		
11:40 – 12:20 40 min	**Sprint 4: Rapid-fire and your handwritten page** **Goal:** Say the core ideas out loud quickly, then capture the few reminders you want on paper.	• Answer the rapid-fire questions out loud; open a hint only after you try. • Write your one page by hand, pen and paper, front and back. Write syntax shapes with generic names (`items`, `setItems`, `Item`, `onAdd`, `ChildProps`) and error → fix pairs, not app-specific code like `addMenuItem`.	• Skim the cheat sheet to choose what goes on your page. • Review Interview craft for how to talk through your process.
12:30 – 1:30 60 min	**Class**		
1:45 – 2:45 60 min	**Sprint 5: Full timed mock #2** **Goal:** Run a fresh problem under time and notice how much smoother the patterns feel now.	• Open Mock 2, start the 25-minute timer, and narrate your steps. • Spend 10–15 minutes reviewing with the rubric. Compare it to Mock 1: what got easier?	• Spend any spare minutes on a short piece of a full build. • If you want one more full run later, Mock 3 is there too.
2:45 – 3:20 35 min	**Sprint 6: TypeScript layer + effects and conditional recap** **Goal:** Add types on top of working JavaScript and refresh the two topics that show up as quick questions.	• Read TypeScript: type aliases, `MenuItem[]`, and function types. • Do the "Fix the bug" set in Drills. • Open a blank Snack and type the 5 Golden Patterns from memory, then run it. • Spend at most 10 minutes on Effects & timers and Promises.	• Skim Conditional rendering. • Try the TypeScript practice: read an error, then fix it.
3:20 – 3:40 20 min	**Light review, then stop coding** Read the cheat sheet and the 5 Golden Patterns once. Open a blank Snack and confirm your Wi-Fi works and you are logged in. Then close the laptop lid, drink water, have a snack, and charge your laptop.		
3:55 5 min early	**Arrive with a charged laptop** Bring your handwritten page if you made one. Take a slow breath. Interview at 4:00 pm. You are ready for this.		

Only 15 minutes? Read the **5 Golden Patterns** and type each one once from memory. They are the core of the whole interview.

The 5 Golden Patterns

> **In one sentence.** Data lives in state, `map` turns it into cards, a new array updates it, a callback lets a child ask for that update, and controlled inputs feed the callback.

Each pattern is short on purpose. If you understand **why** it works, you can rebuild it under pressure instead of trying to recall it letter by letter.

Pattern 1: State from data

Let inference do the typing.

Why it works: State is the memory your screen draws from. When you start `useState` with real data, TypeScript reads the value and works out the type for you, so `menu` becomes `MenuItem[]` with zero extra typing. Only an empty array needs a hint, because TypeScript cannot guess what will go into it. The list lives in `App`; the form's `name` and `popular` state lives in the form itself (Pattern 4).

```
// in App
const [menu, setMenu] = useState(menuItems);   // menu: MenuItem[]
// empty array needs a type:  useState<MenuItem[]>([])
```

Pattern 2: Map data to components

No `{` right after the arrow means the value is returned.

Why it works: `map` calls your arrow function once per item and collects whatever it returns into a new array. When there is no `{` right after `=>`, the expression is returned automatically; the `( )` around the JSX are just grouping so it can span several lines. A `{` right after `=>` starts a function body, so you must write `return` yourself. The `key` lets React tell the cards apart: use a unique id if the data has one (`key={item.id}`); `key={item.name}` is fine only if names are unique.

```
const cards = menu.map((item) => (
  <ItemCard key={item.name} item={item} />
));
// ...then inside JSX:  <ScrollView>{cards}</ScrollView>

// Same thing with a block body: braces need return
const cards2 = menu.map((item) => {
  return <ItemCard key={item.name} item={item} />;
});

// The card itself: typed props, destructured
const ItemCard = ({ item }: ItemCardProps) => (
  <View style={styles.card}>
    <Text>{item.name}</Text>
    {item.popular && <Text>Popular</Text>}
  </View>
);
// item.popular is a boolean, so && is safe (numbers are the danger)
```

Pattern 3: Add to an array immutably

Make a new array; never push into the old one.

Why it works: React re-renders when you hand the setter a **new** array. `[...menu, item]` copies every old item into a fresh array and puts the new one at the end. Because state here is an array, you use square brackets `[ ]`, not curly braces. The updater form is also valid and is safer when the next value depends on the previous one.

```
const addMenuItem = (item: MenuItem) => {
  setMenu([...menu, item]);
  // also valid: setMenu((prev) => [...prev, item]);
};
```

Pattern 4: Callback down, call it up

The parent owns state; the child just asks for changes.

Why it works: Data flows one way: down through props. To change the parent's state, the parent passes a function down as a prop, and the child calls it. The child calls it from a handler that runs on press (not during render) and builds the exact object the type expects. Writing `onPress={onAdd(item)}` would call it during render instead, which can cause "Too many re-renders".

Parent

```
// Parent (App): owns the list
const addMenuItem = (item: MenuItem) => {
  setMenu([...menu, item]);
};
<AddItemForm onAdd={addMenuItem} />
```

Child

```
// Child: owns the text field and switch
const AddItemForm = ({ onAdd }: AddItemFormProps) => {
  const [name, setName] = useState("");
  const [popular, setPopular] = useState(false);
  const handleAdd = () => {
    if (name.trim() === "") return;
    onAdd({ name, popular });
    setName("");
    setPopular(false);
  };
  return (
    <View>
      <TextInput value={name} onChangeText={setName} placeholder="Item name" />
      <Switch value={popular} onValueChange={setPopular} />
      <Pressable onPress={handleAdd}><Text>Add</Text></Pressable>
    </View>
  );
};
```

Pattern 5: Controlled inputs

The input shows state; typing updates state.

Why it works: A controlled input reads its `value` from state and reports every change through a callback. `onChangeText` hands you the new string, so passing `setName` directly works, and so does the arrow version. `Switch` works the same way with a boolean.

```
<TextInput value={name} onChangeText={setName} placeholder="Item name" />
// equivalent: onChangeText={(text) => setName(text)}

<Switch value={popular} onValueChange={setPopular} />
```

Part 1 · Concepts

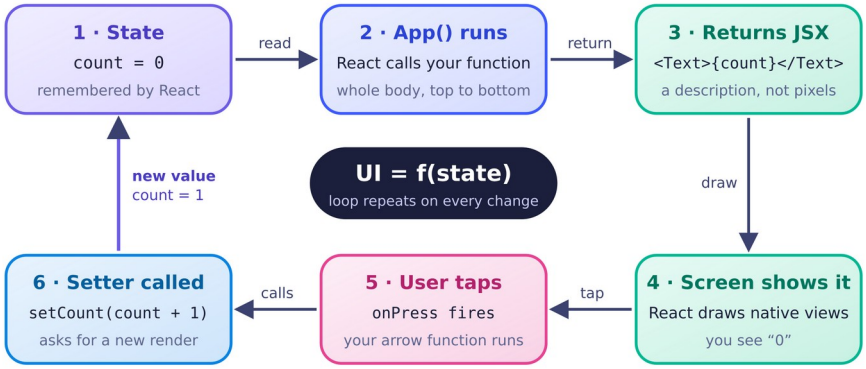
Thirteen topics, each in the same order: a one-sentence takeaway, the essentials in plain English, the key code, a figure where it helps, and the common trap. This is a condensed version of the Concepts page; the interactive explainers live on the website.

1. Mental model

Components are functions React calls

> **In one sentence.** A component is a function that React calls to get JSX; when state changes, React calls it one more time and the screen shows the new result.

When you write `const App = () => { ... }`, you are not drawing the screen yourself. You are writing a **render function**: a recipe that says "given the current data, here is what the screen should look like." React calls that function for you, takes the JSX it returns, and draws real native views on the phone.



The loop you are always living in. Your code only touches steps 2, 3, 5, and 6; React handles the rest.

Why a normal variable does not work

Lecture 3a shows two tempting attempts to remember a count. Both look reasonable, and both break for different reasons. Understanding *why* they break is the whole reason `useState` exists.

✗ Attempt 1 · local let

```
const App = () => {
  let count = 0;
  return (
    <View>
      <Text>{count}</Text>
      <Button title="+1" onPress={() => (count += 1)} />
    </View>
  );
};
```

✗ Attempt 2 · top-level let

```
let count = 0;

const App = () => {
  return (
    <View>
      <Text>{count}</Text>
      <Button title="+1" onPress={() => (count += 1)} />
    </View>
  );
};
```

Common trap: expecting the variable to change right away. Calling `setCount(1)` does not change `count` on the next line. It asks React for a new render, and only *that* render sees the new value. A `console.log(count)` right after the setter prints the old number. See the snapshot simulator in the useState topic.

2. Arrow functions

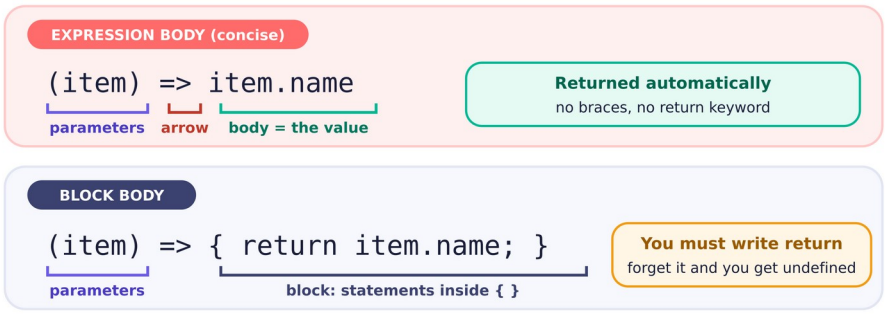
Arrow functions are values you can pass around

In one sentence. An arrow function is a function without a name that you can store or pass like any value; if there is no curly brace right after the `=>`, the expression after it is returned automatically, and if a curly brace follows the `=>`, it starts a function body where you must write `return`.

In JavaScript, **functions are values**, just like numbers and strings. You can put one in a variable, hand it to another function, or give it to a component as a prop. An **anonymous function** is simply a function that does not have its own name. Arrow functions, written with `=>`, are the usual way to write one.

Two body styles: expression vs block

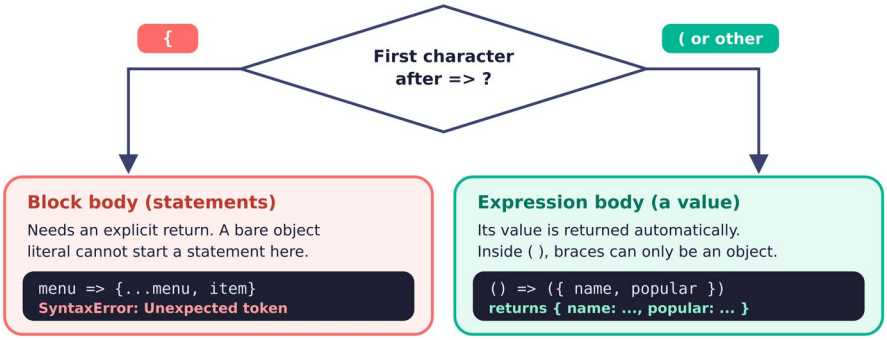
An arrow function has one of two kinds of body. An **expression body** is a single value after the arrow, and that value is **returned automatically** (this is called an *implicit return*). A **block body** starts with a curly brace `{`; it can hold several statements, and it returns `undefined` unless you write `return` yourself.



Same function, two body styles. The very first character after => decides which one JavaScript reads.

Returning an object: wrap it in parentheses

This is the single most common arrow-function surprise. Curly braces have two jobs in JavaScript: they start an **object literal** `{ name: "Pie" }`, and they start a **block of statements**. Right after `=>`, JavaScript always picks **block**. So if you want to return an object from a concise arrow, you must wrap the object in parentheses: `() => ({ name, popular })`. The parentheses tell JavaScript "this is an expression," and inside an expression, braces can only mean an object.



JavaScript decides by looking at one character. That is why `() => ({ ... })` needs the extra parentheses.

✗ Wrong · braces start a block

```
// Intended: copy the menu and add item
setMenu((menu) => {...menu, item});
// "{" after => starts a block body.
// "...menu, item" is not a statement,
// so this is a syntax error.
```

✓ Right · return a new array

```
// State is an ARRAY, so build an array:
setMenu((prev) => [...prev, item]);

// Or the simple direct form:
setMenu([...menu, item]);
```

Passing a function vs calling it

Props like `onPress` want **a function to call later**, when the user taps. There are three shapes you will write, and the difference between them is just a pair of parentheses. Writing a function's name, `handlePress`, **passes** the function. Writing `handlePress()` **calls it right now**, during render, and passes whatever it returns. Wrapping it in an arrow, `() => addItem(item)`, creates a brand-new small function that will call `addItem(item)` later. That last form is how you pass an argument.



Only the middle form runs at the wrong time. If `handlePress` sets state, it triggers a render, which calls it once more, and so on.

✓ Good · pass or wrap

```
// pass the function itself (no argument needed)
<Pressable onPress={handlePress}>

// wrap in an arrow when you need an argument
<Pressable onPress={() => addItem(item)}>

// onChangeText already gives you the text,
// so both of these are equivalent:
<TextInput onChangeText={setName} />
<TextInput onChangeText={(text) => setName(text)} />
```

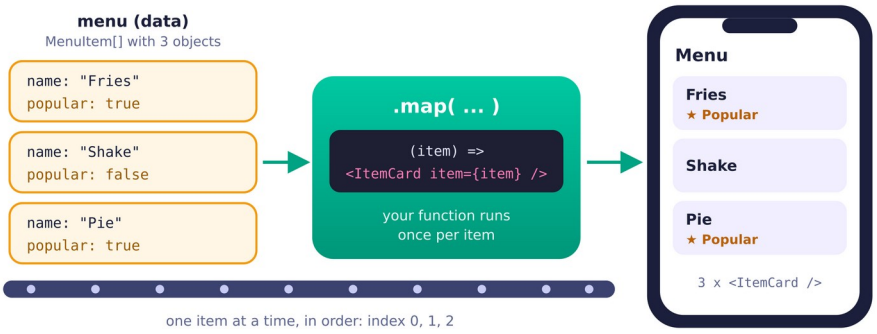
Common trap: onPress={addItem(item)} . It looks like "when pressed, add the item," but the parentheses call `addItem` the instant React renders the component. Read `=>` out loud as "later": `() => addItem(item)` means "*later*, add the item."

3. map & lists

map turns an array of data into an array of components

In one sentence. `menu.map((item) => ( <ItemCard key={item.name} item={item} /> ))` runs your arrow function once per item and gives back a new array of JSX, which React renders when you put it inside `{ }`.

`map` is a method every array has. You give it a function, it calls that function once for each element, and it collects whatever your function **returns** into a **new array** of the same length. The original array is not changed. Lecture 4a introduces it with numbers: `[1, 2, 4, 8, 16].map(x => x * 2)` gives `[2, 4, 8, 16, 32]`.



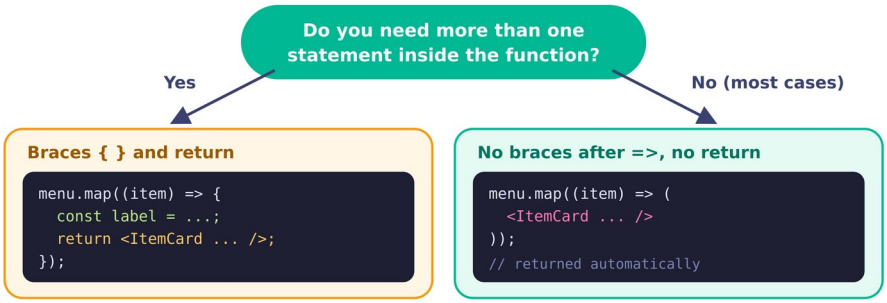
Same number of things in and out: three objects become three cards. `map` never changes `menu`; it builds a fresh array.

The two syntaxes (both correct)

Because `map` takes an arrow function, the two arrow body styles show up here. Most of the time you will use the **no-braces** version: there is no `{` right after `=>`, so the JSX is returned automatically. The parentheses around it are just grouping, so the JSX can span several lines. Switch to the **braces** version only when you need a statement before the JSX, such as computing a label. With braces, the `return` is mandatory.

✓ One-liner · no brace after =>, returned automatically

```
const cards = menu.map((item) => (
  <ItemCard key={item.name} item={item} />
));
```



A one-question decision. If you pick braces, say "return" out loud as you type.

The forgotten-return bug

This bug is silent, which is what makes it nasty. If you open braces and forget `return`, your function still runs for every item, creates the JSX, and then throws it away. Each call returns `undefined`, so `map` hands back `[undefined, undefined, undefined]`. React renders `undefined` as nothing. No red screen, no warning, just an empty list.

✗ Bug · braces, no return

```
const cards = menu.map((item) => {
  <ItemCard key={item.name} item={item} />;
});
// cards is [undefined, undefined, undefined]
// the screen shows no cards
```

✓ Fix · either one

```
// add return
const cards = menu.map((item) => {
  return <ItemCard key={item.name} item={item} />;
});

// or switch the braces to parentheses
const cards = menu.map((item) => (
  <ItemCard key={item.name} item={item} />
));
```

The key prop: a name tag for each item

When a list re-renders, React compares the new array of elements with the previous one. A **`key`** is a stable, unique label that lets React match "this card now" with "that card before," so it can add, remove, or reorder just the right ones. Without keys, React prints the warning `Each child in a list should have a unique "key" prop.`

- **Where it goes:** on the outermost element you return from `map`, for example `<ItemCard key={item.name} ... />`. Not inside `ItemCard`.
- **What to use:** a unique id if the data has one (`key={item.id}`). `key={item.name}` is fine only if names are unique, which is true for the small menus on this page. Never `Math.random()`, which changes every render.
- **Index as key:** `key={index}` silences the warning but can mix up items when you insert, remove, or reorder, so do not use it as your default. It only works safely for a list that never changes.
- **Not a real prop:** `ItemCard` cannot read `props.key`. If the child needs the value, pass it separately.

Where the result goes: a variable or inline

Both patterns render the same thing. Storing the cards in a variable keeps your `return` short and readable, which helps when you are explaining your code out loud. Inline is also common; just mind the extra brackets.

✓ Variable, then {cards}

```
const cards = menu.map((item) => (
  <ItemCard key={item.name} item={item} />
));

return (
  <View style={styles.container}>
    <ScrollView>{cards}</ScrollView>
  </View>
);
```

The index parameter, and filter / find

`map` also passes the position as a second argument, which is handy for numbering. Two cousins come up often: **`filter`** keeps only the items whose function returns `true` (also a new array), and **`find`** returns the first matching item or `undefined`. You can chain them: filter first, then map.

```
// index as the second parameter
const numbered = menu.map((item, index) => (
  <Text key={item.name}>{index + 1}. {item.name}</Text>
));

// only popular items, then cards
const popularCards = menu
  .filter((item) => item.popular)
  .map((item) => <ItemCard key={item.name} item={item} />);

// first match, or undefined
const fries = menu.find((item) => item.name === "Fries");
```

Common trap: braces in filter too. `menu.filter((item) => { item.popular })` returns `undefined` for every item, and `undefined` counts as false, so the result is an empty array. Same rule as `map`: with no brace right after `=>` the value is returned automatically; braces start a body that needs `return`.

4. Objects & spread

Objects, destructuring, and the spread operator

> **In one sentence.** To change an array or object in state, build a **new** one with spread (`[...menu, item]` for arrays, `{ ...item, popular: false }` for objects) instead of editing the old one, because React only re-renders when it sees a new reference.

Objects and how to read them

An **object** groups named values: `const fries = { name: "Fries", popular: true };`. Each name is a **property**. You read one with **dot access**, `fries.name`, or with **bracket access**, `fries["name"]`, which is useful when the property name is in a variable. A menu is just an array of these objects.

Destructuring: unpack by name or by position

Destructuring is a shortcut for pulling values out into variables. Curly braces unpack an object **by property name**. Square brackets unpack an array **by position**. You already use both every day in React: `({ item })` in a component's parameters is object destructuring of the props, and `const [count, setCount] = useState(0)` is array destructuring, because `useState` returns a two-element array.

```
const fries = { name: "Fries", popular: true };

// object destructuring: by NAME
const { name, popular } = fries;      // name = "Fries", popular = true

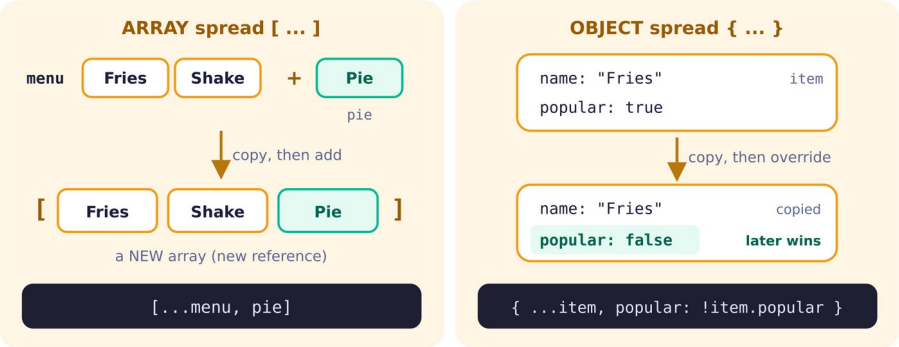
// array destructuring: by POSITION
const pair = ["Fries", "Shake"];
const [first, second] = pair;         // first = "Fries", second = "Shake"

// the same two ideas in React
const ItemCard = ({ item }: ItemCardProps) => <Text>{item.name}</Text>;
const [menu, setMenu] = useState(menuItems);

// shorthand when building an object: { name, popular } means
// { name: name, popular: popular }
const newItem = { name, popular };
```

Spread: copy everything, then add or override

The three dots `...` are the **spread** operator. Inside **square brackets** they copy the elements of an array into a new array. Inside **curly braces** they copy the properties of an object into a new object. After the spread you can add more elements, or override a property. When the same property appears twice, **the later one wins**, so put your override after the spread.



Square brackets make an array; curly braces make an object. Match the brackets to the shape of your state.

Common trap: the wrong brackets. If `menu` is an array, `{ ...menu, item }` does not add to the list. It builds a plain object like `{ 0: fries, 1: shake, item: pie }`, and `menu.map` then crashes because objects do not have `map`. Arrays use `[ ]`; objects use `{ }`. The reverse mistake, `[...someObject]`, throws `TypeError: ... is not iterable`.

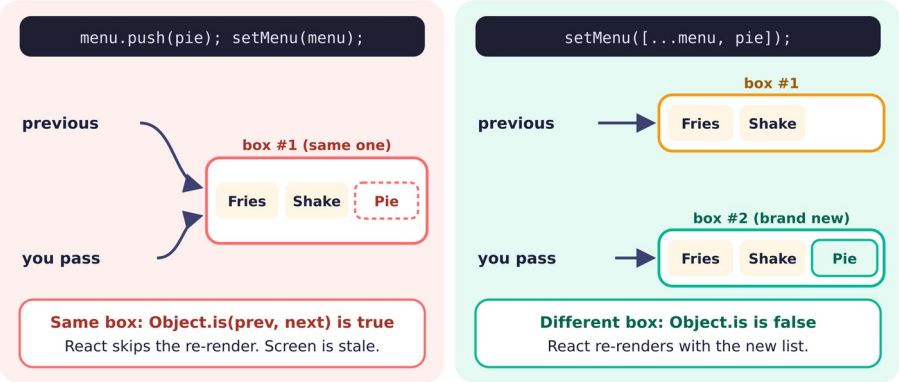
Immutable update recipes

Immutable means "never change the original; make a changed copy." These five recipes cover nearly every list update you will write in the interview. Each one produces a new array, so React sees the change.

Goal	Write this	Avoid (mutates)
Add to the end	`setMenu([...menu, item])`	`menu.push(item)`
Add to the start	`setMenu([item, ...menu])`	`menu.unshift(item)`
Remove one	`setMenu(menu.filter((m) => m.name !== target))`	`menu.splice(i, 1)`
Update one	`setMenu(menu.map((m) => (m.name === target ? { ...m, name: newName } : m)))`	`menu[i].name = newName`
Toggle a boolean	`setMenu(menu.map((m) => (m.name === target ? { ...m, popular: !m.popular } : m)))`	`m.popular = !m.popular`

Why mutation does not re-render

Arrays and objects are stored **by reference**: a variable holds an arrow pointing to a box in memory, not the box itself (Lecture 4b). When you call a setter, React compares the old and new values with `Object.is`. For arrays that asks "is this the *same box*?", not "are the contents the same?" If you `push` into the existing box and pass that same box back, React sees the same reference and **skips the re-render**. A spread creates a **new box**, so React notices.



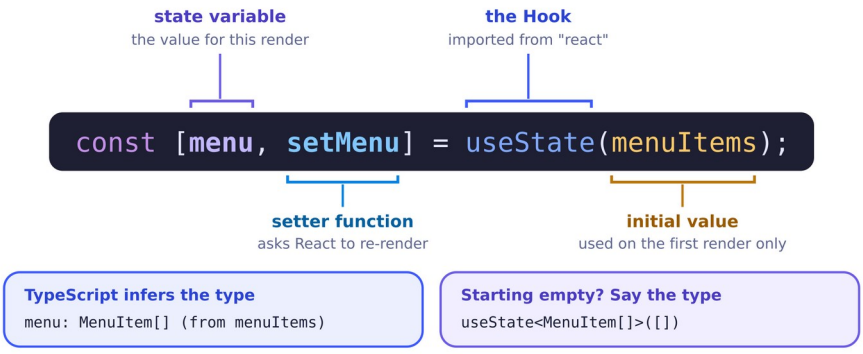
React checks which box, not what is inside it. A new box is the signal that something changed.

5. useState

useState: memory that triggers a re-render

In one sentence. `const [menu, setMenu] = useState(menuItems)` gives you the current value and a setter; calling the setter with a new value (or with an updater function) asks React to re-render, and the new value appears on that next render.

`useState` is a **Hook**: a special React function whose name starts with `use` and which you call at the top of your component. You give it an **initial value**, and it returns a pair that you unpack with array destructuring. The first item is the **state variable** for this render. The second is the **setter**, the only correct way to change it.



Read it left to right: "make a piece of state called `menu`*, changed with* `setMenu`*, starting as* `menuItems`*."*

State is a snapshot

Each render gets its own fixed copy of state, like a photo. Inside one render, `count` never changes, even after you call `setCount`. The setter **queues** a change and asks for a new render; the new value shows up in that next render's snapshot. React also **batches**: it waits until your whole event handler finishes, then re-renders once.

Direct form vs updater form: both are valid

Lecture 5a teaches two ways to call any setter. The **direct form** passes the new value: `setMenu([...menu, item])`. The **updater form** passes a function that receives the previous value and returns the next one: `setMenu((prev) => [...prev, item])`. Both are correct React.

Form	Example	When to reach for it
Direct (simplest default)	`setMenu([...menu, item])` `setName("")`	Most button presses and inputs. Easy to read, easy to explain.
Updater (also valid, safer)	`setMenu((prev) => [...prev, item])` `setSeconds((s) => s + 1)`	When the new value depends on the old one and the snapshot might be stale: several updates in a row, or inside `setInterval` / `setTimeout` callbacks (see effects).

Dissecting a classic broken update

Here is a line that looks close to right but has several separate problems. Suppose the component declared `const [menu, setMenu] = useState(menuItems);`.

✗ Broken

```
setMenuItem(menu => {...menu, item});
```

1. **`setMenuItem`: not the setter you declared.** The setter is `setMenu`. TypeScript reports `Cannot find name 'setMenuItem'`. Always use the exact name from your `useState` line.
2. **`menu =>`: this part is fine.** Passing an updater function is allowed. (Naming its parameter `prev` avoids confusion with the outer `menu`.)
3. **`{` right after `=>`: starts a block body, not an object.** JavaScript now expects statements.
4. **`...menu, item`: not a valid statement.** So the line is a syntax error. Even with parentheses, `({ ...menu, item })` would make an *object*, but `menu` is an *array*.

✓ Fix A · direct form

```
setMenu([...menu, item]);
```

Types: let inference work, add types when it cannot

TypeScript looks at the initial value and **infers** the state type, so you usually write no types at all. The exceptions are starting values that do not say enough: an empty array or `null`.

You write	TypeScript infers	Notes
`useState(0)`	`number`	Counters, prices

You write	TypeScript infers	Notes
`useState("")`	`string`	TextInput values
`useState(false)`	`boolean`	Switch values
`useState(menuItems)`	`MenuItem[]`	Because `menuItems` is typed `MenuItem[]`
`useState([])`	`never[]`	Problem: nothing can be added. Write `useState<MenuItem[]>([])`
`useState<MenuItem | null>(null)`	`MenuItem | null`	"Nothing selected yet"

Where state lives

Put state in **the component that needs it**. If two components need the same data (the form that adds items and the list that shows them), move the state up to their **closest common parent** and pass it down. That idea, called *lifting state up*, is the heart of the callbacks topic. Also remember the Rules of Hooks: call `useState` at the top level of the component, never inside an `if`, a loop, or a handler.

> **Common trap: reading state right after setting it.** `setMenu([...menu, item]); console.log(menu.length);` logs the *old* length. That is expected. If you need the new array right away, store it first: `const next = [...menu, item]; setMenu(next);` and use `next`.

6. Components & props

Custom components and props

> **In one sentence.** A custom component is a capitalized function that receives one **props object** and returns JSX; writing `<ItemCard item={fries} />` makes React call `ItemCard({ item: fries })`.

Once your screen has repeated pieces (a card per menu item, a post per tweet in the Lecture 4b "Twitter" app), you pull each piece into its own **component**. A component is just a function whose name starts with a capital letter. It takes **one argument**, an object called **props** (short for properties), and it returns JSX.



JSX attributes become one props object. Destructuring picks out the pieces you need.

From props to destructured, typed props

Lecture 4a shows the same component two ways. Both are correct JavaScript; the destructured one is what you will see most. In TypeScript, you also describe the shape of props with a **type**, so mistakes like a missing or misspelled prop are caught before you run.

props object

```
const ItemCard = (props: ItemCardProps) => {
  return (
    <View style={styles.card}>
      <Text>{props.item.name}</Text>
    </View>
  );
};
```

JSX rules that save you time

One parent

Default vs named exports

In a multi-file project, each file shares code with **exports**. A file can have **one default export**; the importer picks any name for it and writes no braces. A file can have **many named exports**; the importer must use the exact name, inside braces (Lecture 3a). A common layout is one component per file, exported as default, with shared types exported by name.

components/ItemCard.tsx

```
import { View, Text, StyleSheet } from "react-native";

export type MenuItem = { name: string; popular: boolean };
type ItemCardProps = { item: MenuItem };

const ItemCard = ({ item }: ItemCardProps) => {
  return (
    <View style={styles.card}>
      <Text>{item.name}</Text>
    </View>
  );
};

export default ItemCard;

const styles = StyleSheet.create({
  card: { padding: 12, borderRadius: 10, backgroundColor: "#f0eeff" },
});
```

In the interview Snack. You may be asked to work in a single `App.tsx`. That is completely fine: define the types, then the small components, then `App`, then `styles`, all in one file, and only `App` needs `export default`. If the starter code already has a `components` folder, follow its import style.

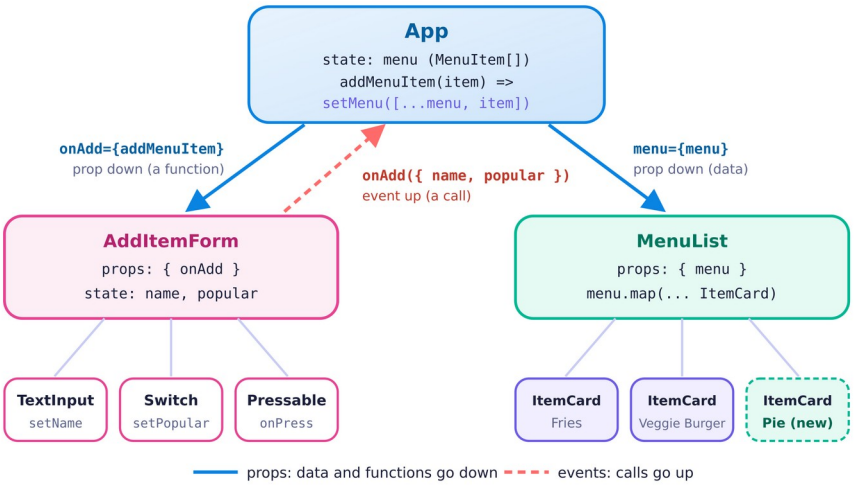
Common trap: forgetting the braces in the parameter. `const ItemCard = (item: ItemCardProps) => ...` names the *whole props object* `item`, so `item.name` does not exist; the name is at `item.item.name`. TypeScript says `Property 'name' does not exist on type 'ItemCardProps'`. Write `({ item }: ItemCardProps)`.

7. Callbacks & data flow

Data flows down, events flow up

In one sentence. Keep the state in the parent, pass both the data and a handler function down as props (`<AddItemForm onAdd={addMenuItem} />`), and let the child call that handler inside an arrow function (`onPress={() => onAdd({ name, popular })}`).

React uses **one-way data flow** (Lecture 4b). A parent can hand data to a child through props, but a child cannot reach up and change its parent's state, and siblings cannot talk to each other directly. So when a form needs to add an item to a list that another component displays, the list's state has to live in their **closest common parent**. Moving it there is called **lifting state up**.



Props carry data and functions down; calls to those functions carry events up.

The pattern in four moves

1. **State in the parent.** `const [menu, setMenu] = useState(menuItems);` lives in `App`, because both the form and the list need it.
2. **A handler in the parent.** `const addMenuItem = (item: MenuItem) => { setMenu([...menu, item]); };` is the only code that changes the menu.
3. **Pass the handler down, without calling it.** `<AddItemForm onAdd={addMenuItem} />`. No parentheses after `addMenuItem`.
4. **The child calls it inside an arrow.** `onPress={() => onAdd({ name, popular })}` builds the object the type expects and sends it up.

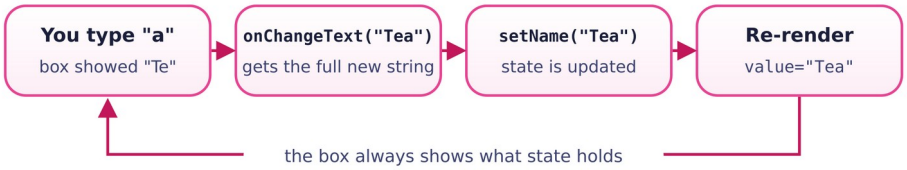
> **Common trap: calling the parent's function by its parent name.** Inside `AddItemForm`, the name `addMenuItem` does not exist; it lives in `App`. The child only knows the prop name, `onAdd`. Writing `addMenuItem(...)` in the child gives `Cannot find name 'addMenuItem'`. Also make sure the child sends a **whole object**, `onAdd({ name, popular })`, not two loose arguments `onAdd(name, popular)`, because the prop type says it takes one `MenuItem`.

8. Inputs

TextInput, Switch, Pressable and Button

> **In one sentence.** An input is "controlled" when state holds its value: you pass the state in with `value`, and you write the new value back into state from `onChangeText` (TextInput) or `onValueChange` (Switch).

Almost every interview handout has a little form: a box to type a name, a switch for a yes/no option such as `popular`, and a button that adds the item to a list. The good news is that **all three use the same idea**. State is the single source of truth, the component only *displays* that state, and every time the user does something, React Native calls a function you gave it so you can update the state.

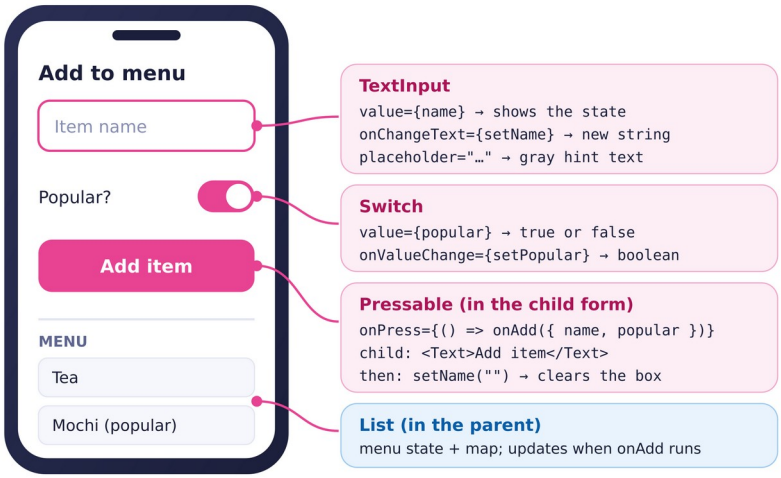


Without onChangeText, state never changes, so the box stays frozen.

The controlled loop. The text box never "remembers" anything by itself; it just mirrors state.

The form, annotated

The picture below is the kind of screen you will see on a handout. Each label on the right tells you which prop drives which part of the screen. Notice that the button lives in the **child** form component, while the list lives in the **parent**. The button talks to the parent by calling the `onAdd` prop.



Every visible part of the form is driven by one prop. If you can label a screenshot like this, you can write the code.

The full pattern, ready for Snack

Paste this into `App.tsx` in Snack. It has the canonical `MenuItem` type, a child form that owns the **draft** values (`name` and `popular`), and a parent that owns the **list**. Read the comments in `handleAdd`: send the object up, then clear the form.

> **Two spellings, same result.** `onChangeText={setName}` and `onChangeText={(text) => setName(text)}` do exactly the same thing. In the first, you hand React Native the setter itself, and it calls `setName("Tea")`. In the second, you hand it a small arrow function that receives `text` and passes it on. Use the arrow version when you need to do something extra, such as `(text) => setName(text.toUpperCase())`. This shortcut works only because `onChangeText` passes exactly the string you want to store. `onPress={setPopular}` is **not** the same idea: `onPress` passes a press event object, so you would store an event instead of a boolean. Write `onPress={() => setPopular(true)}`.

Each component in one table

Component	Shows	Change prop	Your function receives	Notes
`TextInput`	`value` (string)	`onChangeText`	the full new string	`placeholder` shows gray hint text while the box is empty.
`Switch`	`value` (boolean)	`onValueChange`	the new boolean	Controlled: if you never update state, it snaps back.
`Pressable`	its children	`onPress`	a press event (you usually ignore it)	Words must be inside a `<Text>` child. `style` may be a function of `{ pressed }`.
`Button`	`title` (string)	`onPress`	a press event	Also `disabled` and `color`. It has **no** `style` prop.

Pressable or Button?

`Button` is the quick option: give it a `title` and an `onPress` and you are done. It looks like the platform's default button, and you cannot style it, because it has no `style` prop. Even `color` means different things: it sets the text color on iOS but the background color on Android. That is why many people reach for **`Pressable`** when the handout shows a custom-looking button. Pressable is just a box that reacts to presses, so you style it like a `View` and put a `<Text>` inside.

✓ **Quick: Button**

```
<Button
  title="Add item"
  onPress={handleAdd}
  disabled={name === ""}
  color="#e84393"
/>
// needs: import { Button } from "react-native";
```

The four bugs that show up most

✗ **Bug: wrong event name**

```
<TextInput
  value={name}
  onChange={setName}
/>
// onChange gets an EVENT object, not a string
```

✓ **Works**

```
<TextInput
  value={name}
  onChangeText={setName}
/>
// onChangeText gets the new string
```

Common trap.

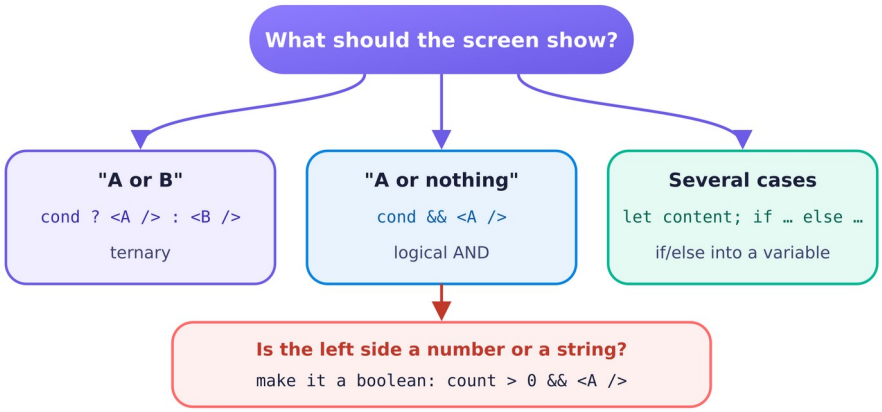
- **`onChange` instead of `onChangeText`.** `onChange` hands you an event object, so your state fills up with an object instead of a string. In the interview, always reach for `onChangeText`.
- **`value={name}` with no change handler.** The box is controlled but nothing ever updates `name`, so it looks frozen.
- **Forgetting to clear.** After `onAdd(...)`, call `setName("")` (and `setPopular(false)`) so the form is empty for the next item. Because `value={name}`, clearing state clears the box.
- **`onPress={onAdd({ name, popular })}`.** The parentheses call the function right now, during render. Wrap it: `onPress={() => onAdd({ name, popular })}`. If the call sets state, you get "Too many re-renders".
- **Bare words inside Pressable.** `<Pressable>Add</Pressable>` breaks, because text must be inside `<Text>`.

9. Conditional rendering

Showing different things based on state

In one sentence. Pick the tool by the shape of the choice: `cond ? <A /> : <B />` for "A or B", `cond && <A />` for "A or nothing" (with a real boolean on the left), and an `if`/`else` into a variable when the logic gets longer.

JSX is just JavaScript values, so "conditional rendering" means **choosing which value to put on the screen**. You already know the tools from plain JavaScript; the only new thing is that the values happen to be components.



Three tools, chosen by the shape of the decision. The red box is the one React Native rule that catches people.

1. if/else into a variable (Lecture 3b)

This is the pattern from the shopping cart lecture. You declare a variable, fill it with different JSX in each branch, and then drop it into your return with curly braces. It is the most **readable** choice when each branch has several lines, and it is a great fallback in the interview because it is plain JavaScript you already know. Here is the "SOLD OUT" activity: once the cart button is tapped, it becomes disabled and its label changes.

2. The ternary: "A or B" inside JSX

When only a word or a style changes, a ternary keeps everything in one place. Read `soldOut ? "SOLD OUT" : "ADD TO CART"` out loud as "if sold out, then SOLD OUT, otherwise ADD TO CART". This version does the same job as the one above in fewer lines.

✓ **Works · same button with a ternary**

```
<Pressable
  style={[styles.button, soldOut && styles.disabled]}
  disabled={soldOut}
  onPress={() => setSoldOut(true)}
>
  <Text style={styles.buttonText}>{soldOut ? "SOLD OUT" : "ADD TO CART"}</Text>
</Pressable>
```

3. Logical AND: "A or nothing"

`cond && <A />` renders `<A />` when `cond` is true and **nothing** when it is `false`. React skips `false`, `null` and `undefined`. The catch is that `&&` does not return `false` when the left side is falsy; it returns **the left side itself**. So if the left side is the number `0` or an empty string `""`, that value lands on the screen, outside any `<Text>`.

You write	What gets rendered
`{true && <Badge />}`	the Badge
`{false && <Badge />}`	nothing
`{cart.length && <Badge />}` when the cart is empty	the number `0`, as a bare string: in React Native this gives the error *"Text strings must be rendered within a <Text> component."*
`{cart.length > 0 && <Badge />}`	nothing when empty, the Badge otherwise

✗ **Bug: 0 leaks onto the screen**

```
{cart.length && (
  <Text>Items in cart: {cart.length}</Text>
)}
```

✓ **Works: a real boolean**

```
{cart.length > 0 && (
  <Text>Items in cart: {cart.length}</Text>
)}
```

Empty-state message

A handout often says "if there are no items, show a friendly message". That is "A or B", so it is a ternary. Build your mapped list first, then choose between it and the message.

✓ Works · empty state

```
const cards = menu.map((item) => (
  <ItemCard key={item.name} item={item} />
));

return (
  <ScrollView>
    {menu.length === 0 ? (
      <Text style={styles.empty}>No items yet. Add one above!</Text>
    ) : (
      cards
    )}
  </ScrollView>
);
```

Conditional styles

A `style` prop can take an **array**, and later entries win. Combine that with `&&` to add a highlight only when something is true. Here a falsy entry (`false`) is simply skipped, so the card keeps its normal look. This one is safe because `item.popular` is a boolean, and styles are not text, so nothing leaks onto the screen.

✓ Works · highlight popular items

```
<View style={[styles.card, item.popular && styles.popularCard]}>
  <Text style={styles.name}>{item.name}</Text>
  {item.popular && <Text style={styles.tag}>Popular</Text>}
</View>

// in StyleSheet.create:
// card:    { padding: 12, borderRadius: 12, backgroundColor: "#fff" },
// popularCard: { borderWidth: 2, borderColor: "#e84393" },
```

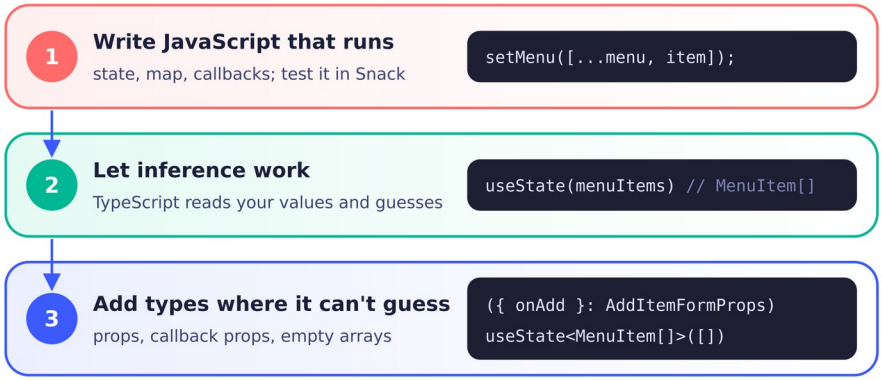
> **Common trap.** `{count && <Text>…</Text>}` renders `0` when `count` is zero, and React Native reports *"Text strings must be rendered within a <Text> component."* Use `{count > 0 && …}` or a ternary that returns `null`. Also remember that `if` statements cannot go directly inside JSX curly braces; use a ternary there, or put the `if` above the `return`.

10. TypeScript

Types on top of working JavaScript

> **In one sentence.** Write JavaScript that runs, let TypeScript infer what it can, and then add types in the three places it cannot guess: function parameters and props, callback props, and empty arrays or `null` starting values.

TypeScript is a superset of JavaScript. Any JavaScript you write is also valid TypeScript *syntax* (the checker may still complain about it); TypeScript only adds optional labels, called **types**, that describe what shape a value has. A tool called the type checker reads those labels and underlines code that does not match. In Snack, those are the red squiggles. They are warnings from the checker, not crashes: Snack strips the types before running, so the app often still runs with a squiggle on screen. A red error screen in the preview, on the other hand, is always a real runtime error.



Layers 1 and 2 cover most of your code. Layer 3 is a short list of places you can memorize.

Describing shapes: type alias vs interface

A **type alias** (`type`) gives a name to any type. An **interface** gives a name to an object shape. For the objects you will write in the interview, they are interchangeable, so pick one and be consistent. This kit uses `type`, because it can also name things an interface cannot, such as a union of strings.

✓ type alias

```
type MenuItem = {
  name: string;
  popular: boolean;
};
```

Question	`type`	`interface`
Describe an object?	Yes	Yes
Name a union like `"small" | "large"`?	Yes	No
Syntax	`type X = { … };` (has `=`)	`interface X { … }` (no `=`)
Extend another shape	`type B = A & { … }`	`interface B extends A { … }`

The building blocks

You only need a handful of pieces. **Primitives** are `string`, `number` and `boolean`. An **array** of a type is that type followed by `[]`, so a list of menu items is `MenuItem[]`. A question mark after a property name makes it **optional**, meaning it may be missing. A **union of literals** such as `"small" | "large"` allows only those exact strings.

✓ Works · every building block in one type

```
type Drink = {
  name: string;               // primitive: text
  price: number;              // primitive: any number
  iced: boolean;              // primitive: true or false
  toppings: string[];         // array of strings
  size?: "small" | "large";   // optional, and only these two strings
};

const order: Drink = { name: "Matcha", price: 5, iced: true, toppings: [] };
// size is optional, so leaving it out is fine.
// size: "medium" would be an error: not one of the allowed strings.
```

Function types: describing callbacks

A **function type** looks like an arrow function with types instead of a body. Read `(item: MenuItem) => void` as "a function that takes one `MenuItem` and returns nothing useful". `void` means "returns nothing". A callback with no inputs is `() => void`. The parameter needs a **name** as well as a type: in `(MenuItem) => void`, TypeScript reads `MenuItem` as the parameter's *name* with no type, and reports `Parameter has a name but no type. Did you mean 'arg0: MenuItem'?`

✓ Works · typing props, including callbacks

```
type MenuItem = { name: string; popular: boolean };

type ItemCardProps = { item: MenuItem };
type AddItemFormProps = { onAdd: (item: MenuItem) => void };
type ClearButtonProps = { onClear: () => void };

const ItemCard = ({ item }: ItemCardProps) => (
  <Text>{item.name}</Text>
);

const AddItemForm = ({ onAdd }: AddItemFormProps) => {
  const [name, setName] = useState("");
  const [popular, setPopular] = useState(false);
  // onAdd must receive a MenuItem, so build exactly that object:
  return (
    <Pressable onPress={() => onAdd({ name, popular })}>
      <Text>Add</Text>
    </Pressable>
  );
};
```

useState and the empty array

When you write `useState(menuItems)`, TypeScript looks at `menuItems` and infers `MenuItem[]`. When you write `useState([])`, there is nothing to look at, so TypeScript infers `never[]`: "an array that can never hold anything". The first time you try to put an item in it, you get an error like the one below. The fix is to tell `useState` the type in angle brackets. That angle-bracket slot is called a **generic**, or type argument.

✗ Squiggle: never[]

```
const [menu, setMenu] = useState([]);

setMenu([...menu, item]);
// Type 'MenuItem' is not assignable
// to type 'never'.
```

✓ Works: give the type

```
const [menu, setMenu] = useState<MenuItem[]>([]);

setMenu([...menu, item]); // fine

// Starting from real data? No type needed:
// useState(menuItems) is already MenuItem[]
```

Where do I need annotations?

Place	Annotate?	Example
Component props	**Yes**	`({ item }: ItemCardProps)`
Parameters of a function you define on its own line	**Yes**	`const addMenuItem = (item: MenuItem) => { … }`
`useState` with a real starting value	No, inferred	`useState("")`, `useState(false)`, `useState(menuItems)`
`useState` with `[]` or `null`	**Yes**	`useState<MenuItem[]>([])`
Inline callbacks (`map`, `onPress`, `onChangeText`)	No, contextual	`menu.map((item) => …)`
Return types	Optional	Inferred from what you return; add one only if you want the checker to hold you to it.

Reading the five errors you are most likely to see

TypeScript errors look scary because they are long, but they follow a pattern: **"X is not assignable to Y"** means "you gave me X where I expected Y". Read the last line first; it is usually the most specific.

You see	Plain English	Usual fix
`Type 'MenuItem' is not assignable to type 'never'.` (it appears on the line where you add to the array)	Your state was created from an empty `[]`, so TypeScript thinks it must stay empty.	`useState<MenuItem[]>([])`
`Property 'onAdd' is missing in type '{}' but required in type 'AddItemFormProps'.`	You rendered a component without one of its required props.	Pass the prop (`onAdd={addMenuItem}`), or mark it optional with `?` if it really is.
`Parameter 'item' implicitly has an 'any' type.`	A function on its own line has a parameter with no type, and there is no context to guess from.	`(item: MenuItem) => …`
`Property 'nmae' does not exist on type 'MenuItem'.`	A typo, or a field your type does not have.	Fix the spelling, or add the field to the type.
`Type 'string' is not assignable to type 'boolean'.`	You passed the wrong kind of value, for example `popular="yes"`.	Pass the right type: `popular={true}`.

Common trap. Trying to write all the types first and getting stuck before anything renders. In a 25-minute interview, a working screen with a squiggle is far better than perfect types and a blank screen. Get the JavaScript running, then add types to props, callbacks and empty arrays. And if you see `never` anywhere in an error, look for a `useState([])`.

11. Layout

StyleSheet, the box model and Flexbox

In one sentence. In React Native every `View` is a flex container whose default direction is **column**, so `justifyContent` moves children along that main axis (vertically by default) and `alignItems` moves them along the cross axis.

StyleSheet and style arrays

Styles in React Native are JavaScript objects with camelCase names: `backgroundColor`, not `background-color`. Sizes are plain numbers, measured in density-independent pixels, not strings like `"12px"`. You collect them with `StyleSheet.create`, which keeps them out of your render code and lets the editor check property names. A `style` prop can also take an **array**; when two entries set the same property, the **last one wins**.

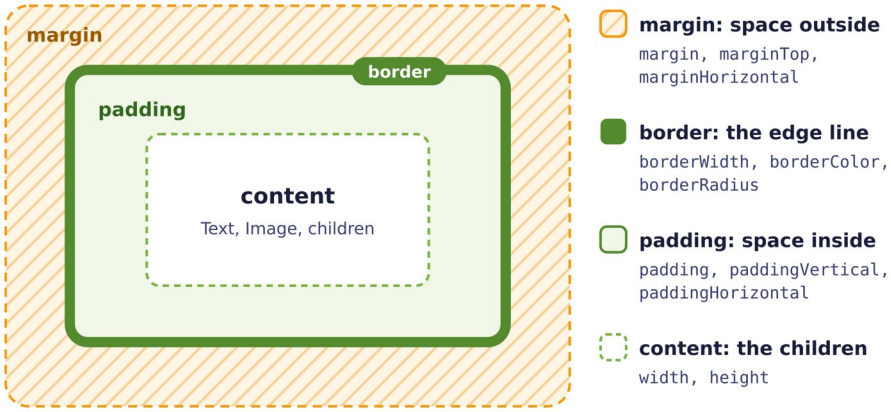
✓ Works

```
const styles = StyleSheet.create({
  card: { padding: 12, borderRadius: 12, backgroundColor: "#fff" },
  popularCard: { backgroundColor: "#fdeef6" },
});

<View style={[styles.card, item.popular && styles.popularCard]} />
// popular items: the second entry overrides backgroundColor
```

The box model

Every component is a box with four layers, from the inside out: the **content**, the **padding** (space inside the border), the **border**, and the **margin** (space outside, pushing neighbors away). In React Native, a `width` or `height` includes the padding and the border, so adding padding never makes the box bigger than the size you set.

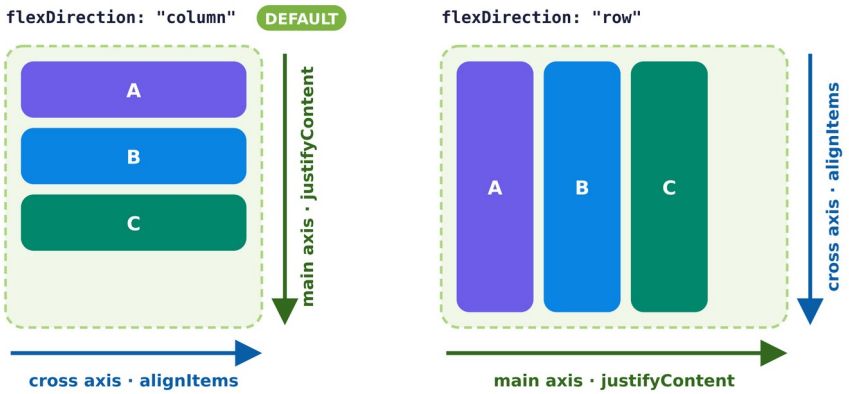


In RN, width and height include padding and border.

Padding pushes content inward; margin pushes neighbors outward. A card with `padding: 12` and `marginBottom: 10` is the most common combination.

Flexbox: two axes

Flexbox is the system that arranges a container's direct children. It always has two axes. The **main axis** is the direction children are stacked in, set by `flexDirection`. The **cross axis** is perpendicular to it. **`justifyContent` works on the main axis; `alignItems` works on the cross axis.** In React Native the default `flexDirection` is `"column"`, unlike the web, where it is `"row"`. So in a plain `View`, `justifyContent: "center"` centers things *vertically*. Switch to `"row"` and the axes swap.



The boxes stretch across the cross axis because `alignItems` defaults to `"stretch"`. Give a child a fixed width (in a column) or height (in a row) and the stretch stops.

flex: 1, flexWrap and gap

- **`flex: 1`** on a child means "take up the leftover space along the main axis". On your top-level container it means "fill the whole screen". Siblings with `flex: 1` share the space evenly; `flex: 2` gets twice the share. A parent with no size and no `flex` has zero height, so its `flex` children can vanish.
- **`flexWrap: "wrap"`** on the container lets children move onto a new line when they run out of room, which is how you make a simple grid of tiles.
- **`gap`** on the container adds space *between* children, so you do not need a margin on each one. `rowGap` and `columnGap` set each direction separately.

Recipe: the card row

Most list screens in a handout are built from the same card: a picture or emoji on the left, a column of text in the middle, and a button on the right. Here is the recipe. The card is a **row**; the middle part gets `flex: 1` so it soaks up the extra space and pushes the button to the right edge.

ScrollView or FlatList?

A plain `View` does not scroll, so a long list just runs off the bottom of the screen. **`ScrollView`** is the simple fix: put your mapped cards inside it. It renders every child at once, which is perfect for the short lists in an interview. **`FlatList`** only renders what is on screen, so it is better for long lists. It does the mapping for you: you give it the array as `data`, a function that draws one item as `renderItem`, and a function that returns a unique string for each item as `keyExtractor`.

✓ ScrollView + map

```
<ScrollView contentContainerStyle={{ padding: 16 }}>
  {menu.map((item) => (
    <ItemCard key={item.name} item={item} />
  ))}
</ScrollView>
```

Common trap. Expecting web defaults. In React Native, children stack **top to bottom** unless you write `flexDirection: "row"`, and when you switch to a row, `justifyContent` and `alignItems` swap jobs. If something "won't center", say out loud which axis is the main axis right now, then pick the property. Second trap: putting `flexDirection` on the child. Flex properties that arrange children go on the **container** (the direct parent).

12. Effects

Hooks, useEffect, timers and Promises

In one sentence. `useEffect(setup, deps)` runs your `setup` function **after** React draws the screen, the dependency array decides when it runs next, and the function you `return` cleans up (for example, `clearInterval`) before the next run and when the component goes away.

The rules of hooks

A **hook** is a special React function whose name starts with `use`: `useState`, `useEffect`, and so on. React keeps track of hooks by the **order** they are called in, so that order must be identical on every render. That gives three rules:

1. **Call hooks at the top level** of your component, before any early `return`, and never inside an `if`, a loop, or an event handler.
2. **Call hooks only from components** (or from your own custom hooks), not from ordinary helper functions.
3. **Hook names start with `use`.** That is how you and React can tell them apart from normal functions.

✗ Bug: hook inside a handler

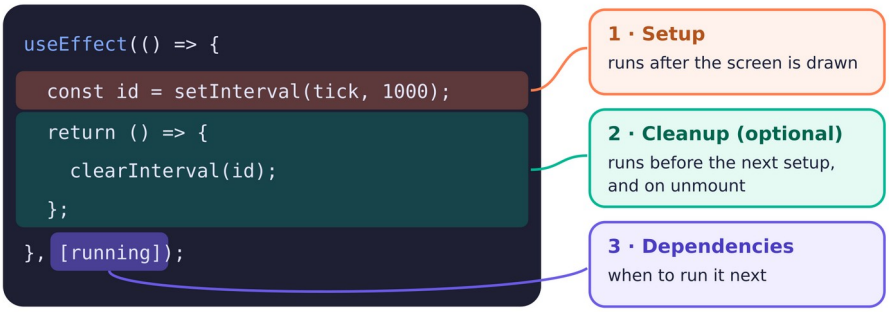
```
const handleAdd = () => {
  const [name, setName] = useState("");
  // "Invalid hook call"
};
```

✓ Works: top of the component

```
const AddItemForm = () => {
  const [name, setName] = useState("");
  const handleAdd = () => { /* use name here */ };
  // ...
};
```

Anatomy of useEffect

An **effect** is code that keeps your component in sync with something outside React, such as a timer or a server. You do not call it yourself; React calls it after the screen has been drawn. It has three parts.



Setup starts something, cleanup stops it, and the dependency array decides how often that cycle happens.

The dependency array

You write	The effect runs...	Typical use
`useEffect(fn)` (no array)	after **every** render	Rarely what you want. Setting state in here can loop forever.
`useEffect(fn, [])`	**once**, after the first render	Start a clock, fetch data when the screen opens.
`useEffect(fn, [a, b])`	after the first render, and whenever `a` or `b` changed	Start or stop a timer when `running` changes; refetch when a search term changes.

setTimeout once, setInterval repeatedly

Both take a **callback function** and a delay in **milliseconds** (1000 ms = 1 second), and both return an ID you use to cancel them. `setTimeout` fires one time. `setInterval` keeps firing until you call `clearInterval(id)`.

The timer example (writing sprint app)

This is the shape of the writing sprint timer from Lecture 5a: a countdown that starts and pauses. The effect depends on `running`. When `running` turns true, setup starts an interval. When it turns false (or the screen closes), the cleanup clears the old interval first.

Why the updater form matters here: the stale closure

A **closure** is a function that remembers the variables around it at the moment it was created. The interval's callback is created once, during the render where the effect ran, so it remembers `seconds` **from that render**. With an empty `[]` dependency array, the effect never runs a second time, so the callback never sees a newer value. It keeps computing `60 - 1` forever, and the clock gets stuck on 59.

✗ **Bug: stuck at 59**

```
useEffect(() => {
  const id = setInterval(() => {
    setSeconds(seconds - 1); // seconds is always 60 here
  }, 1000);
  return () => clearInterval(id);
}, []);
```

✓ **Works: updater form**

```
useEffect(() => {
  const id = setInterval(() => {
    setSeconds((s) => s - 1); // React passes the latest value
  }, 1000);
  return () => clearInterval(id);
}, []);
```

Promises: values that arrive later

A **Promise** is an object that stands for a result that is not ready yet, such as data from a server. It starts out **pending**. Later it either **resolves** (succeeds, with a value) or **rejects** (with an error). You do not wait for it; you attach a callback with `.then`,

and that callback runs when the value arrives. In Lecture 5b, `setTimeoutAsync(1000)` was an imagined Promise version of `setTimeout`, used to show this idea; it is not a built-in function.

✓ Works · .then and .catch

```
console.log("fetching data");
fetchData()
  .then((data) => {
    console.log("here is your data:", data); // runs later, when data is ready
  })
  .catch((error) => {
    console.log("something went wrong:", error); // runs if the Promise rejects
  });
console.log("this line prints BEFORE the data arrives");
```

Fetch on mount (planet explorer)

"Load the data when the screen opens" is an effect with an empty dependency array. This example has a pretend `fetchData` that resolves after one second, so it runs in Snack without a real server.

Common trap.

- **No cleanup on an interval.** Each new run of the effect starts another interval, and the clock speeds up. Always `return () => clearInterval(id)`.
- **Missing dependency array plus a state update.** `useEffect(() => setCount(count + 1))` runs after every render, which causes a render, which runs the effect… forever.
- **`setSeconds(seconds - 1)` inside an interval with `[]`.** The stale closure freezes the clock. Use `setSeconds((s) => s - 1)`.
- **Expecting data on the first render.** The effect runs after the first render, so the screen must handle the empty starting state.

13. Interview craft

How to work during the 25 minutes

In one sentence. The interviewer is watching your **process**, so read the handout, sketch the component tree, build one small step at a time in plain JavaScript, run it often, read every error, and talk while you work.

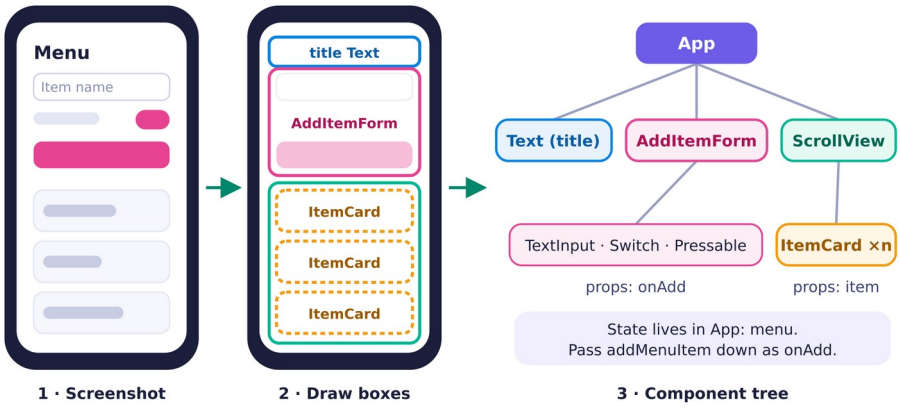
You are not expected to finish the whole problem. The handout gives you a reference screenshot and a step-by-step breakdown, and the TA may ask you to move on to a different step. What earns credit is showing that you can **break a screen into components, wire up state and props, lay it out with Flexbox, and debug calmly**. That is a skill you can rehearse, just like syntax.

1. Read the handout and decompose

1. **Read every step once, quickly,** before writing code. Note which steps are layout, which are state, and which are callbacks.
2. **Find the data.** What is the array, and what does one item look like? Write the `type` in your head (or in a comment): `{ name: string; popular: boolean }`.
3. **Find the state.** What changes on screen? Each changing thing is a `useState`. Where does it live? The lowest component that every user of it sits under.
4. **Find the events.** Which taps or keystrokes change that state? Each one is a handler, defined where the state lives and passed down as a prop.

2. Sketch the component tree first

Boxes on the screenshot become components in your code. Thirty seconds of sketching saves five minutes of rewriting.



Each colored box becomes a component. Repeated boxes (the cards) mean `map`. The box that contains everything that changes is where state lives.

3. JavaScript first, run often

Build in thin slices and look at the preview after every one. A good order is: **static layout with hard-coded text → map the real data → add state → add the handler and callback → add types**. If something breaks, you know it was the last small thing you changed. Snack updates the preview as you type, so glance at it constantly.

4. Debug with console.log

When you are not sure what a value is, print it. Snack shows logs in the panel at the bottom of the editor. Label each log so you know which line printed it.

✓ Works · labeled logs

```
const addMenuItem = (item: MenuItem) => {
  console.log("addMenuItem got:", item);  // did the child send the right object?
  setMenu([...menu, item]);
  console.log("menu right after set:", menu); // still the OLD array: state is a snapshot
};

console.log("render, menu has", menu.length, "items"); // runs every render
```

5. Think aloud

Talking lets the TA see your reasoning and point out a wrong assumption early. It feels strange at first, so rehearse a few sentences until they come naturally.

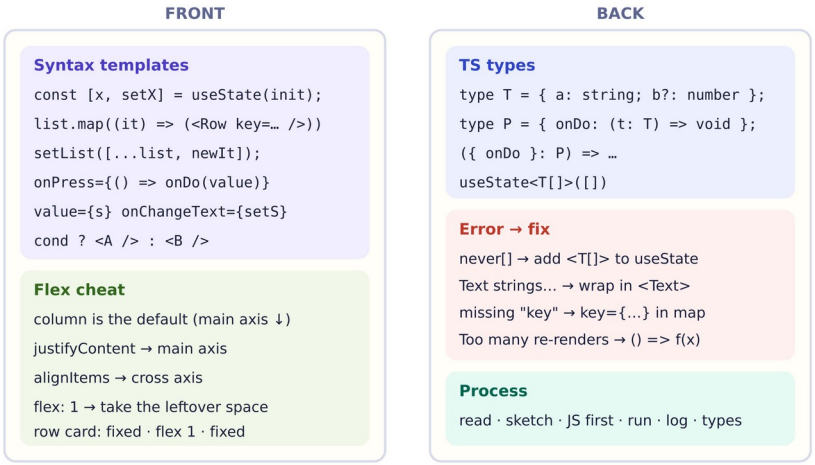
6. Search for knowledge, not solutions

The internet (including the Google Search AI overview) is allowed. Lecture 4a gives a clear rule of thumb: **search for general knowledge, not hints or solutions.**

Shows you are resourceful	Shows over-reliance
Searching documentation: "what values work for justifyContent?"	Asking for the solution: "how do I center the links in a navbar?"
Debugging syntax: "what is the syntax for a self-closing tag in React Native?"	Pattern matching: copying a snippet from your notes that looks similar and tweaking it until it runs
Reviewing a concept: "what's the difference between justifyContent and alignItems?"	
Looking up an error: pasting the exact error message	

7. Design your one handwritten page

You may bring **one page of handwritten notes, front and back**, in English, with pen and paper only (no tablets or screenshots), and you hand it in at the end. Notes are optional, and you do not need them to do well. They must **not** contain homework solutions or solution-like code. So write **syntax shapes** with generic names (`items`, `setItems`, `Item`, `onAdd`, `ChildProps`, or even `T` and `list`) plus **error → fix pairs**, rather than app-specific code such as `addMenuItem` or `ItemCard`. Writing the page is itself one of the best study sessions you can do.



Generic names keep it a pattern sheet, not a solution.

A suggested layout. Big, clear handwriting beats cramming: you want to find a pattern in two seconds while someone is watching.

A pacing plan for 25 minutes

Minutes	What you do
0–3	Read the handout, sketch the tree, say your plan out loud.
3–20	Work step by step. After each step, look at the preview and say what works.
20–25	Tidy up, add missing types, and explain what you would do next if you had more time.

> **Common trap.** Going silent and typing for five minutes, then discovering the first line was wrong. Narrate as you go, run after every small change, and if you are stuck for more than a minute, say what you have tried and what you suspect. That is exactly the process the interview is designed to see.

Part 2 · Practice

Practice in three layers: plain JavaScript first (state, map, arrow functions), then arrow functions as callbacks between components, then TypeScript types on top. Attempt each one before you open a hint. Answers to every drill and the solutions to every exercise are in the Answer Key at the end of this guide.

Quick drills

Predict the output

D1. A local variable vs. state

You press the button **three times**. What does the first line on screen show? Decide before you reveal.

```
export default function App() {
  let count = 0;
  const [score, setScore] = useState(0);

  return (
    <View>
      <Text>count: {count}, score: {score}</Text>
      <Pressable
        onPress={() => {
          count = count + 1;
          setScore(score + 1);
        }}
      >
        <Text>Press me</Text>
      </Pressable>
    </View>
  );
}
```

D2. One-liner map

What does this print or show? Decide before you reveal.

```
const doubled = [1, 2, 4].map((x) => x * 2);
console.log(doubled);
```

D3. Braces but no return

What does this print or show? Decide before you reveal.

```
const doubled = [1, 2, 4].map((x) => {
  x * 2;
});
console.log(doubled);
```

D4. Spread makes a copy (Lecture 4b)

What does this print or show? Decide before you reveal.

```
const array = [1, 2, 4, 8, 16];
const array2 = [...array, 32];
console.log(array);
console.log(array2);
```

D5. Same array, two names

What does this print or show? Decide before you reveal.

```
const array = [1, 2, 4];
const array2 = array;
array2.push(8);
console.log(array);
console.log(array === array2);
```

D6. Three setter calls in one press

What does this print or show? Decide before you reveal.

```
// count is 0 before the press
<Pressable
  onPress={() => {
    setCount(count + 1);
    setCount(count + 1);
    setCount(count + 1);
  }}
>
  <Text>+3?</Text>
</Pressable>
```

Fix the bug

D7. Adding an item to an array in state

Something is wrong here. Say what it is, then write the fix.

✗ Buggy

```
const [menu, setMenu] = useState(menuItems);

const addMenuItem = (item: MenuItem) => {
  setMenu(menu => {...menu, item});
};
```

D8. The list renders nothing

Something is wrong here. Say what it is, then write the fix.

✗ Buggy

```
const cards = menu.map((item) => {
  <ItemCard item={item} />
});
```

D9. The app freezes with a render error

Something is wrong here. Say what it is, then write the fix.

✗ Buggy

```
<Pressable onPress={addItem(item)}>
  <Text>Add</Text>
</Pressable>
```

D10. The state changes but the screen doesn't

Something is wrong here. Say what it is, then write the fix.

✗ Buggy

```
const addMenuItem = (item: MenuItem) => {
  menu.push(item);
  setMenu(menu);
};
```

D11. The name becomes an object

Something is wrong here. Say what it is, then write the fix.

✗ Buggy

```
<TextInput value={name} onChange={setName} placeholder="Item name" />
```

D12. The yellow warning about keys

Something is wrong here. Say what it is, then write the fix.

✗ Buggy

```
{menu.map((item) => (
  <ItemCard item={item} />
))}
```

D13. Works with items, errors at zero

Something is wrong here. Say what it is, then write the fix.

✗ Buggy

```
{count && <Text>{count} items in cart</Text>}
```

D14. Not assignable to type 'never'

Something is wrong here. Say what it is, then write the fix.

✗ Buggy

```
const [menu, setMenu] = useState([]);

const addMenuItem = (item: MenuItem) => {
  setMenu([...menu, item]); // error: ... not assignable to type 'never'
};
```

D15. An arrow that should return an object

Something is wrong here. Say what it is, then write the fix.

✗ Buggy

```
const makeItem = () => { name: "Tea", popular: false };
```

D16. A setter that doesn't exist

Something is wrong here. Say what it is, then write the fix.

✗ Buggy

```
const [count, setCount] = useState(0);

<Pressable onPress={() => setState(count + 1)}>
  <Text>+1</Text>
</Pressable>
```

Fill the blank

D17. Map to cards

Type the missing code in each box, then press Check. Spacing and quote style don't matter.

```
const cards = menu.____((item) => (
  <ItemCard ____={item.name} item={item} />
));
```

D18. Add without mutating

Type the missing code in each box, then press Check. Spacing and quote style don't matter.

```
const addMenuItem = (item: MenuItem) => {
  setMenu([ ____menu, item ]);
};
```

D19. A controlled TextInput and Switch

Type the missing code in each box, then press Check. Spacing and quote style don't matter.

```
<TextInput value={name} ____={setName} placeholder="Item name" />
<Switch value={popular} ____={setPopular} />
```

D20. Child calls the parent's callback

Type the missing code in each box, then press Check. Spacing and quote style don't matter.

```
<Pressable onPress={____ onAdd({ name, ____ })}>
  <Text>Add</Text>
</Pressable>
```

D21. Toggle one field

Type the missing code in each box, then press Check. Spacing and quote style don't matter.

```
setMenu(menu.map((item) =>
  item.name === target ? { ____item, popular: ____ } : item
));
```

D22. Types for an empty list and a callback prop

Type the missing code in each box, then press Check. Spacing and quote style don't matter.

```
const [menu, setMenu] = useState<____>([]);

type AddItemFormProps = { onAdd: (item: MenuItem) => ____ };
```

Exercises by topic

The highest-yield exercises, at least one for every topic. Open a blank Snack at snack.expo.dev, rename the file to App.tsx, and build it. The starter files are on the Practice page; the full solutions are in the Answer Key.

Fruit cart: pass a function, don't call it

Core · 12 min · Paste into Snack

Goal. Show one button per fruit. Pressing a fruit adds its price to the total. Add a **Clear** button that resets the total to 0. The key skill is giving `onPress` a **function value** like `() => addToCart(fruit.price)`.

> **Hint 1.** `onPress` wants a function that React Native will call later. `addToCart(fruit.price)` is not a function; it is the *result* of calling one, right now, during render.

> **Hint 2.** Wrap the call in an anonymous arrow function: `onPress={() => addToCart(fruit.price)}`. Build the buttons with `fruitList.map((fruit) => ( <Pressable key={fruit.emoji} ...> ... </Pressable> ))`.

Playlist cards with a block-body map

Core · 12 min · Paste into Snack

Goal. Finish a `SongCard` component that shows the position and title in bold, the artist underneath, and a **Liked** badge only for liked songs. Then map the songs to cards using a **block body** that computes `position = index + 1` before it returns.

> **Hint 1.** `map` passes two arguments to your callback: the item and its index. With braces you can declare variables first, but the last line must be `return <SongCard ... />;`.

> **Hint 2.** In `SongCard`, use `{song.liked && ( <View style={styles.badge}><Text style={styles.badgeText}>Liked</Text></View> )}`. `liked` is a boolean, so `&&` is safe here. In `App`: `songs.map((song, index) => { const position = index + 1; return <SongCard key={song.title} song={song} position={position} />; })`.

Four immutable recipes: add, prepend, remove, toggle

Warm-up · 8 min · Paste into Snack

Goal. Without changing the original `habits` array, build four **new** arrays: one with a habit added at the end, one with it added at the start, one with habit 3 removed, and one where habit 2 has `done` flipped. The screen prints all five so you can check your work.

> **Hint 1.** Adding uses array spread `[...]`. Removing uses `filter`, which keeps the items for which your arrow returns `true`. Toggling uses `map`, which returns one item for every item.

> **Hint 2.** `[...habits, newHabit]`, `[newHabit, ...habits]`, `habits.filter((habit) => habit.id !== 3)`, and `habits.map((habit) => habit.id === 2 ? { ...habit, done: !habit.done } : habit)`.

Snapshots and the updater form

Interview-level · 12 min · Paste into Snack

Goal. A **Read 3 chapters** button calls `readOne()` three times, but the count only goes up by 1. Fix it with the **updater form** of the setter. Then predict what the "Right after setting" line shows, and explain why in a comment.

> **Hint 1.** During one press, `chapters` is a fixed number from this render (a **snapshot**). All three calls compute `0 + 1`. You need a form of the setter that receives the latest queued value instead.

> **Hint 2.** `setChapters((c) => c + 1);` React queues the three updater functions and runs them in order: 0 to 1, 1 to 2, 2 to 3. For TODO 2, the log shows the number *before* this press, because `chapters` doesn't change until the next render.

Extract a PlantCard component

Warm-up · 8 min · Paste into Snack

Goal. All the card JSX currently lives inside `App`'s `map`. Move it into its own component, `PlantCard`, that takes a `plant` prop. Type the props, destructure them, and use `<PlantCard key={...} plant={plant} />` in the map.

> **Hint 1.** A component is just a function that takes **one** argument, the props object, and returns JSX. Its name must start with a capital letter.

> **Hint 2.** `type PlantCardProps = { plant: Plant };` and `const PlantCard = ({ plant }: PlantCardProps) => ( <View style={styles.card}> ... </View> );`. Keep `key` in `App`, on `<PlantCard>`.

Playlist: an AddSongForm that calls onAdd

Core · 15 min · Paste into Snack

Goal. Build `AddSongForm`, a child with two TextInputs (title, artist) and an **Add song** button. On press, it calls `onAdd({ title, artist, liked: false })`, building the full object the `Song` type expects, and then clears its inputs. The parent's `addSong` appends to the list.

> **Hint 1.** Three pieces must line up: the props type says `onAdd` takes a `Song`; the parent defines `addSong(song: Song)` and passes it as `onAdd={addSong}`; the child calls `onAdd(...)` with an object that has **all three** fields.

> **Hint 2.** `type AddSongFormProps = { onAdd: (song: Song) => void };` In the child: `const handleAdd = () => { onAdd({ title, artist, liked: false }); setTitle(""); setArtist(""); };` and `<Pressable onPress={handleAdd}>`. In the parent: `const addSong = (song: Song) => { setSongs([...songs, song]); };`.

Habit tracker: clear the input after adding, disable Add when empty

Core · 14 min · Paste into Snack

Goal. Typing a habit and pressing **Add** appends it to the list and **clears the input**. The Add button is **disabled** (and looks grey) while the trimmed text is empty, so blank habits can't be added. Bonus: pressing the keyboard's return key also adds.

> **Hint 1.** Clearing a controlled input means setting its state back to `""`. Disabling is a derived boolean computed from that same state.

> **Hint 2.** `const canAdd = label.trim().length > 0;` After `setHabits(...)`, call `setLabel("");`. On the Pressable: `disabled={!canAdd}` and `style={[styles.button, !canAdd && styles.buttonDisabled]}`. Bonus: `onSubmitEditing={addHabit}` on the TextInput, plus `if (!canAdd) return;` at the top of `addHabit`.

Café: the 0 && trap

Core · 10 min · Paste into Snack

Goal. This menu shows a banner with the number of popular items. It works until you press **Clear popular flags**, which sets every count to 0 and the app shows a red error. Find the bug and fix it so the banner hides at 0 and a muted "No popular items yet" line shows instead.

> **Hint 1.** `a && b` doesn't return `true` or `false`. It returns `a` itself if `a` is falsy. What is `popularCount` after clearing, and where would it be rendered?

> **Hint 2.** Make the left side a real boolean, or use a ternary: `{popularCount > 0 ? ( <View style={styles.banner}>...</View> ) : ( <Text style={styles.muted}>No popular items yet</Text> )}`. The per-item `item.popular && ...` is fine because `popular` is already a boolean.

JS first, types second: type a working app

Core · 14 min · Paste into Snack

Goal. This plant app is written like plain JavaScript. It mostly runs, but TypeScript shows red squiggles. Add the types: a `Plant` type, props types for both components, a function type for `onAdd`, a type argument for the **empty** `useState([])`, and a parameter type for `addPlant`.

> **Hint 1.** Start from the data: write `type Plant` first. Everything else refers to it. Then fix the errors in the order Snack lists them; one fix often removes several squiggles.

> **Hint 2.** `type PlantCardProps = { plant: Plant };`, `type AddPlantFormProps = { onAdd: (plant: Plant) => void };`, `useState<Plant[]>([])`, and `const addPlant = (plant: Plant) => { ... }`. Inline callbacks like `(plant) =>` inside `plants.map` need no annotation; they are inferred.

Café: header, scrolling list, pinned footer

Core · 10 min · Paste into Snack

Goal. Fill in the empty styles so the screen has a header at the top, a list that takes **all the remaining space** and scrolls, and a footer pinned to the bottom. Each row puts the name on the left and a badge on the right, vertically centered.

> **Hint 1.** In React Native, `flexDirection` defaults to **column**, so children stack top to bottom. `flex: 1` makes a child grow to fill the leftover space along that direction.

> **Hint 2.** `container: { flex: 1, paddingTop: 60, paddingHorizontal: 20, paddingBottom: 30, backgroundColor: "#f6f7fc" }`, `list: { flex: 1 }`, `row: { flexDirection: "row", justifyContent: "space-between", alignItems: "center", ... }`, and the same three for `footer`.

Writing sprint: a countdown timer with useEffect and cleanup

Core · 15 min · Paste into Snack

Goal. Build a 60-second countdown with **Start**, **Pause**, and **Reset**. Use `useEffect` with `[running]` as the dependency array to start a `setInterval` when running, and **return a cleanup** that clears it. Use the **updater form** inside the interval. When the time reaches 0, stop automatically.

> **Hint 1.** An effect runs **after** a render. Its dependency array says when to re-run: `[running]` means "after the first render and whenever `running` changes." Whatever function the effect returns is the cleanup, which React calls before re-running and on unmount.

> **Hint 2.** `useEffect(() => { if (!running) return; const id = setInterval(() => { setSecondsLeft((s) => (s > 0 ? s - 1 : 0)); }, 1000); return () => clearInterval(id); }, [running]);` and a second effect: `useEffect(() => { if (secondsLeft === 0) setRunning(false); }, [secondsLeft]);`.

Build A: Café Menu Builder

Interview-level · 25 min · Mirrors the interview format. A complete guided build: do each step, check the checkpoint, then move on.

The handout. Build a single-screen app for a café. It shows the current menu as a list of `ItemCard`s, each with a **Popular** badge when the item is popular, and a count like "3 items · 2 popular". Below the list, an `AddItemForm` child has a TextInput for the name, a Switch for popular, and an **Add item** button that calls `onAdd({ name, popular })`. Bonus: each card can toggle popular and be removed.

> **Do it in two passes.** First build each step in plain JavaScript style, letting inference work. Then add the types listed at the end. The reference screenshot on the right is what the finished app should roughly look like.

Step 1: Type, data, and state

Do: Write the `MenuItem` type and a starting array. In `App`, put the array in state. Render just a title and the raw count so you can see something.

> **Hint.** Start the state from data, `useState(menuItems)`, and inference types it as `MenuItem[]` for free.

Checkpoint: what you should see now. The title and "3 items". No red errors. If `setMenu` is unused, that's fine for now.

Step 2: ItemCard + map inside a ScrollView

Do: Create `ItemCard` with props `{ item: MenuItem }`. In `App`, map the menu to cards with a one-liner and put the result in a `ScrollView`.

> **Hint.** `const cards = menu.map((item) => ( <ItemCard key={item.name} item={item} /> ));` then `<ScrollView style={styles.list}>{cards}</ScrollView>`. No braces after `=>`, so the card is returned automatically.

Checkpoint: what you should see now. Three white cards with the item names, and no key warning in the Snack console.

Step 3: Popular badge styling

Do: Give popular cards an amber left border and a small **Popular** badge on the right.

> **Hint.** Style arrays merge styles: `style={[styles.card, item.popular && styles.cardPopular]}`. For the badge, `item.popular` is a boolean, so `{item.popular && ( ... )}` is safe.

Checkpoint: what you should see now. Latte and Banana bread have an amber stripe and a Popular pill; Chai doesn't.

Step 4: Derived count

Do: Show "3 items · 2 popular". Compute the popular count from `menu` during render; don't add state for it.

> **Hint.** `const popularCount = menu.filter((item) => item.popular).length;`

Checkpoint: what you should see now. The line reads "3 items · 2 popular".

Step 5: AddItemForm with controlled inputs

Do: Create `AddItemForm`. It keeps its own `name` and `popular` state, with a controlled TextInput and Switch, and a Pressable labeled **Add item**. Don't wire the button yet. Render `<AddItemForm />` under the list.

> **Hint.** `<TextInput value={name} onChangeText={setName} placeholder="Item name" />` and `<Switch value={popular} onValueChange={setPopular} />`.

Checkpoint: what you should see now. You can type in the field and flip the switch. Pressing Add does nothing yet.

Step 6: Wire the callback: parent handler, child calls onAdd

Do: In `App`, write `addMenuItem(item: MenuItem)` that appends immutably, and pass it down as `onAdd`. Type the form's props. In the child, call `onAdd({ name, popular })` on press, then clear the inputs and disable the button while the name is empty.

> **Hint.** Parent: `setMenu([...menu, item])` and `<AddItemForm onAdd={addMenuItem} />`. Child: `type AddItemFormProps = { onAdd: (item: MenuItem) => void };` and `onPress={() => onAdd({ name, popular })}` to start. Then grow it into a `handleAdd` that also clears.

Checkpoint: what you should see now. Type "Mocha", flip Popular on, press Add item: a new card appears with a badge, the count updates to "4 items · 3 popular", and the inputs clear.

Step 7: Bonus: toggle popular and remove

Do: Give `ItemCard` two more callback props, `onTogglePopular(name)` and `onRemove(name)`, with small buttons. Implement both in `App` with `map` + spread and `filter`. Also show an empty-state line when the menu is empty, and ignore duplicate names, since names are the keys.

> **Hint.** Toggle: `menu.map((m) => m.name === name ? { ...m, popular: !m.popular } : m)`. Remove: `menu.filter((m) => m.name !== name)`.

Checkpoint: what you should see now. Mark popular / Unmark flips the badge and the count; Remove deletes the card. Remove everything and the empty-state message appears.

Types added after the JavaScript pass

Here is every place a type went in, and why. Notice how many lines needed **nothing**, because inference handled them.

Where	What was added	Why
`type MenuItem = { name: string; popular: boolean }`	Describes the data once.	Every other type refers to it, and typos like `item.nmae` get caught.
`const menuItems: MenuItem[] = [...]`	Annotates the starting data.	Lets `useState(menuItems)` infer `MenuItem[]`, so the state line needs no type.
`useState(menuItems)`, `useState("")`, `useState(false)`	Nothing; inferred.	The initial value already says the type. Only an empty `[]` (or `null`) needs `useState<T>()`.
`type ItemCardProps` + `({ item, ... }: ItemCardProps)`	Props object type on the destructured parameter.	Without it: "Binding element 'item' implicitly has an 'any' type." With it, a missing prop is an error at the call site.
`type AddItemFormProps = { onAdd: (item: MenuItem) => void }`	A function type for the callback prop.	Forces the child to send a complete `MenuItem`; forgetting `popular` is caught immediately.
`const addMenuItem = (item: MenuItem) => ...`	Parameter annotation on a standalone function.	TypeScript can't infer parameters of a function defined on its own (TS7006 "implicitly has an 'any' type").
`(name: string)` on `togglePopular` and `removeItem`	Parameter annotations.	Same reason; they also match the `(name: string) => void` props they are passed to.
`(item) =>` inside `menu.map`, `(m) =>` inside `filter`	Nothing; contextually typed.	Inline callbacks get their parameter types from the array they're called on.

The complete final App.tsx for Build A is in the Answer Key.

Part 3 · Mock Interviews

> **In one sentence.** You get 25 minutes, a Snack and a handout; the TA watches how you plan, code, debug and talk, so a clear process matters more than a finished app.

The 25-minute game plan

Each phase builds on the last, and the app **runs at the end of every phase**. If the TA moves you on, you still have working code to talk about. The times are a guide, not a rule.

Before typing, draw the boxes and arrows. Ask two questions: **where does each piece of state live?** and **which way does each piece of data travel?**

How to run a mock on yourself

1. **Pick one mock and do not scroll past its handout.** The parts, hints and solutions sit below it.
2. **Open a blank Snack** at snack.expo.dev, rename the file to `App.tsx`, and paste the starter code. Check that it runs.
3. **Press Start on the mock's timer.** It stays pinned at the top while you scroll through that mock, and it turns red at 5:00 left.
4. **Spend the first two minutes reading and sketching** on paper. Then work through the parts in order, and **say your plan out loud**. Talking to an empty room feels odd, but it is the skill being observed.
5. **Stuck for about two minutes?** Open Hint 1, then Hint 2, then Hint 3. If one part blocks you for five minutes, move to the next part, just as a TA might ask you to.
6. **At 25:00, stop typing.** Tick the rubric honestly, then read the model solutions and the "Why this works" notes. Finish with the review routine in After a mock.

A 60-second talk-aloud script

Say something close to this in your first minute, before you type. It shows the TA your plan, and it settles your nerves.

1. **Read:** "Let me read every part first, so I know where this is going."
2. **Data:** "The data is an array of `MenuItem`s, so I'll keep it in state in `App` with `useState(menuItems)`."
3. **List:** "I'll `map` each item to an `ItemCard` inside a `ScrollView`, with a key on each card."
4. **Form:** "The form owns its own text and switch state. The list lives in `App`, so `App` passes an `onAdd` callback down and the form calls it."
5. **Order:** "I'll get it on screen in plain JavaScript first and run it after each step, then add the types. Starting with Part 1."

> **If the interviewer says "move on":** stub the part with hard-coded values and keep going. For example, render one `<ItemCard item={{ name: "Test", popular: true }} />` instead of the `map`, or have the button call `onAdd({ name: "Test", popular: false })`. Say "I'll stub this and come back to it," and come back if time remains. The later parts still get something real to work with.

Snack quirks worth knowing

- **Red squiggles usually do not stop the app.** They are TypeScript type errors; Snack still runs your code. That is why "JS first, types second" works: get it running, then fix the squiggles.
- **A red error screen is a runtime error.** Read the message and the line number on it first (for example `App.tsx:42`), then look at that line.
- **The Web preview can behave differently from a phone.** A string outside `<Text>`, or a stray `0` from `count && …`, may only crash on iOS or Android. Check the iOS or Android tab before you call a part done.

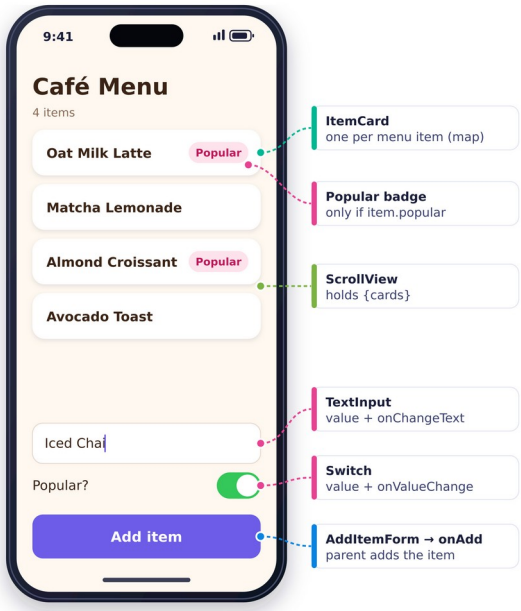
- `onChangeText={setName}` **is fine**, because it passes the new string. `onPress={setX}` **is not the same**: Pressable passes an event object, so your state would become that event. Write `onPress={() => setX(…)}`.
- **Passing vs calling:** `onPress={onAdd(item)}` calls `onAdd` during render, which can cause "Too many re-renders." Pass a function instead: `onPress={() => onAdd(item)}`.

> **Common trap.** Coding silently for ten minutes, then running the app for the first time. Run after every small step. A red screen you get one minute after a change points straight at that change.

Mock 1: Café Menu

> **In one sentence.** A typical Interview-2-style app: render a list from state with `map`, then let a child form add to it by calling a callback the parent passed down.

The handout: Café Menu



You are building a tiny ordering screen for a campus café. The starter file already has the `MenuItem` type, four menu items, and two empty components. Your job is to show the menu, mark popular items, and let the user add a new item with a name and a **Popular?** switch.

Given type

```
type MenuItem = {
  name: string;
  popular: boolean;
};
```

Step-by-step breakdown

1. Show every item in `menuItems` as a card in a scrolling list.
2. Build `ItemCard` so it shows the name, plus a **Popular** badge for popular items.
3. Build `AddItemForm` with a text input, a **Popular?** switch, and an **Add item** button.
4. Make **Add item** add the new item to the list and clear the input.
5. Stretch: show the item count, and disable the button while the input is empty.

Parts, hints and clarifying questions

Part 1: Render the menu with map (about 6 min)

> **Your task.** In `App`, put `menuItems` into state. Turn each item into an `<ItemCard />` with `map`, and render the cards inside a `ScrollView` under the title. Pass each card its item as a prop called `item`. You will see a red squiggle on `item={item}` until Part 2 gives `ItemCard` a props type. That is expected, and the app still runs.

> **Clarifying question you could ask.** "Is each item's `name` unique, so I can use it as the `key`?" (Here, yes. If names could repeat, you would ask whether items have an `id`.)

Hint 1 · Nudge. You need two things in `App`: a state variable that holds the array, and an array of JSX built from that state.

Hint 2 · Pointer. `useState(menuItems)` gives you `[menu, setMenu]`, and TypeScript infers `menu: MenuItem[]` from the data. `menu.map(...)` returns a **new array** of whatever your arrow function returns, so make the arrow return an `<ItemCard />`.

Hint 3 · Almost there

```
const cards = menu.map((item) => (
  <ItemCard key={___} item={___} />
));
// in the JSX:
<ScrollView style={styles.list}>{cards}</ScrollView>
```

Common trap. Writing `menu.map((item) => { <ItemCard item={item} /> })`. The braces start a function body with no `return`, so every call returns `undefined` and **nothing renders, with no error**. Either swap the braces for parentheses or add `return`.

Part 2: ItemCard with typed props and a Popular badge (about 5 min)

Your task. Give `ItemCard` a typed `item` prop. Show the item's name. Show a small **Popular** badge only when `item.popular` is `true`.

Clarifying question you could ask. "Should the badge sit on the same row as the name, or under it?" (The screenshot shows the same row, pushed to the right.)

Hint 1 · Nudge. Props arrive as **one object**. What does that object look like when the parent writes `<ItemCard item={item} />`?

Hint 2 · Pointer. Describe the props object with a type, `type ItemCardProps = { item: MenuItem };`, then destructure it in the parameter list. For the badge you want "badge or nothing", which is exactly what `&&` does.

Hint 3 · Almost there

```
const ItemCard = ({ item }: ItemCardProps) => {
  return (
    <View style={styles.card}>
      <Text>{item.name}</Text>
      {item.popular && ( /* badge here */ )}
    </View>
  );
};
```

Common trap. Typing the parameter as `({ item: MenuItem })`. Inside destructuring, a colon means **rename**, so that line makes a variable called `MenuItem` and leaves `item` untyped. Put the type after the closing brace: `({ item }: ItemCardProps)` or `({ item }: { item: MenuItem })`.

Part 3: AddItemForm with TextInput and Switch (about 5 min)

Your task. Inside `AddItemForm`, add a `TextInput` for the item name, a `Switch` labelled **Popular?**, and an **Add item** button (a `Pressable` with a `Text` inside). Both inputs must be **controlled**: their current values live in state. Render `<AddItemForm />` below the ScrollView. The button does not need to do anything yet.

Clarifying question you could ask. "Can the form keep its own `name` and `popular` state, or do you want that state in `App`?" (In the form is fine, because only the form needs it until **Add** is pressed.)

Hint 1 · Nudge. Each input needs **a value to show** and **a function to call when the user changes it**. Where can a value live so that changing it re-renders the form?

Hint 2 · Pointer. Use two pieces of state in the form: a string and a boolean. `TextInput` takes `value` + `onChangeText`. `Switch` takes `value` + `onValueChange`.

Hint 3 · Almost there

```
const [name, setName] = useState("");
const [popular, setPopular] = useState(false);

<TextInput value={name} onChangeText={setName} placeholder="New item name" />
<Switch value={popular} onValueChange={setPopular} />
```

Common trap. Writing `<Pressable>Add item</Pressable>` with a bare string. On iOS and Android that gives "Text strings must be rendered within a <Text> component." The Web tab may hide this bug, so always wrap the label in `<Text>`.

Part 4: Make Add item work with a callback (about 7 min)

Your task. Define `addMenuItem(item: MenuItem)` in `App`. Pass it to `AddItemForm` as a prop called `onAdd`. When **Add item** is pressed, the form calls `onAdd` with a new `MenuItem` built from `name` and `popular`, then clears the text input.

Clarifying question you could ask. "After adding, should the **Popular?** switch also reset to off?" (Either is fine. Asking shows you are thinking about the user.)

Hint 1 · Nudge. Which component owns `menu`? **Only the component that owns a piece of state can change it**, so the code that adds to the menu must live there. How can the form reach it?

Hint 2 · Pointer. A parent can hand a child a **function** as a prop. Type that prop as a function type, `type AddItemFormProps = { onAdd: (item: MenuItem) => void };`. The child calls it with an object of the `MenuItem` shape.

Hint 3 · Almost there

```
// App (parent)
const addMenuItem = (item: MenuItem) => {
  setMenu(___);            // a NEW array: old items + item
};
<AddItemForm onAdd={addMenuItem} />

// AddItemForm (child)
onPress={() => onAdd({ ___, ___ })}
```

Common trap. `setMenu(menu => {...menu, item})` has two separate bugs. First, `{` right after `=>` starts a **function body**, not an object, so nothing is returned. Second, `menu` is an **array**, so the copy needs square brackets: `[...menu, item]`. Fixed: `setMenu((prev) => [...prev, item])`. Also call the setter you declared (`setMenu`), not a generic `setState`, and keep the arrow: `onPress={onAdd({ name, popular })}` calls `onAdd` during render instead of on tap.

Stretch: item count and a disabled Add button (if time)

> **Your task.** Show "4 items" under the title and keep it correct as items are added. Disable **Add item** (and fade it) while the name is empty.

> **Hint 1 · Nudge.** A number you can compute from state should be **derived** during render, not stored in more state.

> **Hint 2 · Pointer.** `menu.length` is always current. For the button, compute a boolean such as `canAdd` from `name`, then use the `disabled` prop and a style array.

> **Hint 3 · Almost there**
>
> ```
> <Text style={styles.count}>{menu.length} items</Text>
> const canAdd = name.trim().length > 0;
> ```

The starter App.tsx for each mock is on the Mock Interview page; paste it into a blank Snack before you start the timer.

Complete final solution · Mock 1: Café Menu

One file, ready to paste into Snack. It includes the stretch goals. In the form, the Part 4 arrow function is given a name, `handleAdd`, and it trims the name before sending it up.

✓ App.tsx · complete solution

```
import { useState } from "react";
import { View, Text, TextInput, Pressable, ScrollView, Switch, StyleSheet } from "react-native";

type MenuItem = {
  name: string;
  popular: boolean;
};

type ItemCardProps = { item: MenuItem };
type AddItemFormProps = { onAdd: (item: MenuItem) => void };

const menuItems: MenuItem[] = [
  { name: "Oat Milk Latte", popular: true },
  { name: "Matcha Lemonade", popular: false },
  { name: "Almond Croissant", popular: true },
  { name: "Avocado Toast", popular: false },
];

// Part 2: a presentational child. It only READS its props.
const ItemCard = ({ item }: ItemCardProps) => {
  return (
    <View style={styles.card}>
      <Text style={styles.cardName}>{item.name}</Text>
      {item.popular && (
        <View style={styles.badge}>
          <Text style={styles.badgeText}>Popular</Text>
        </View>
      )}
    </View>
  );
};

// Parts 3 + 4: the form owns its own input state, and reports
// a finished MenuItem UP to the parent through onAdd.
const AddItemForm = ({ onAdd }: AddItemFormProps) => {
  const [name, setName] = useState("");
  const [popular, setPopular] = useState(false);
  const canAdd = name.trim().length > 0; // Stretch: disable when empty
```

```
  const handleAdd = () => {
    onAdd({ name: name.trim(), popular }); // build the object the type expects
    setName("");                           // clear the input
    setPopular(false);
  };

  return (
    <View style={styles.form}>
      <TextInput
        style={styles.input}
        value={name}
        onChangeText={setName}
        placeholder="New item name"
      />
      <View style={styles.switchRow}>
        <Text style={styles.switchLabel}>Popular?</Text>
        <Switch value={popular} onValueChange={setPopular} />
      </View>
      <Pressable
        style={[styles.addButton, !canAdd && styles.addButtonDisabled]}
        onPress={handleAdd}
        disabled={!canAdd}
      >
        <Text style={styles.addButtonText}>Add item</Text>
      </Pressable>
    </View>
  );
};

export default function App() {
  // Part 1: state from data. TypeScript infers MenuItem[].
  const [menu, setMenu] = useState(menuItems);

  // Part 4: the handler lives where the state lives (the parent).
  const addMenuItem = (item: MenuItem) => {
    setMenu([...menu, item]); // or: setMenu((prev) => [...prev, item]);
  };

  // Part 1: one-liner map. No { after =>, so the JSX is returned.
  const cards = menu.map((item) => (
    <ItemCard key={item.name} item={item} />
  ));

  return (
    <View style={styles.container}>
      <Text style={styles.title}>Café Menu</Text>
      <Text style={styles.count}>{menu.length} items</Text>
      <ScrollView style={styles.list}>{cards}</ScrollView>
      <AddItemForm onAdd={addMenuItem} />
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, paddingTop: 60, paddingHorizontal: 20, backgroundColor: "#fff8f0" },
  title: { fontSize: 30, fontWeight: "800", color: "#3b2416" },
  count: { fontSize: 15, color: "#8a6a55", marginBottom: 12 },
  list: { flex: 1 },
  card: {
    backgroundColor: "#ffffff",
    borderRadius: 14,
    padding: 16,
    marginBottom: 10,
    flexDirection: "row",
    justifyContent: "space-between",
    alignItems: "center",
```

```
  },
  cardName: { fontSize: 18, fontWeight: "600", color: "#3b2416" },
  badge: { backgroundColor: "#ffe3ec", borderRadius: 999, paddingHorizontal: 10, paddingVertical:
   4 },
  badgeText: { color: "#c2185b", fontWeight: "700", fontSize: 13 },
  form: { paddingVertical: 16, gap: 10 },
  input: {
    backgroundColor: "#ffffff",
    borderWidth: 1,
    borderColor: "#e8d8cc",
    borderRadius: 12,
    padding: 12,
    fontSize: 16,
  },
  switchRow: { flexDirection: "row", justifyContent: "space-between", alignItems: "center" },
  switchLabel: { fontSize: 16, color: "#3b2416" },
  addButton: { backgroundColor: "#6c5ce7", borderRadius: 12, paddingVertical: 14, alignItems:
   "center" },
  addButtonDisabled: { opacity: 0.4 },
  addButtonText: { color: "#ffffff", fontWeight: "700", fontSize: 16 },
});
```

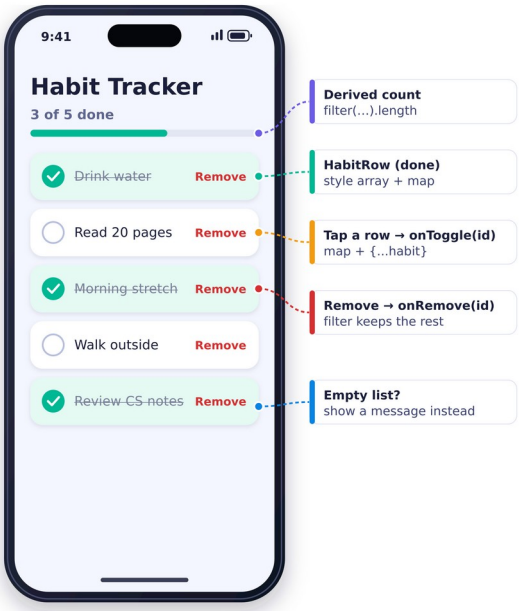
Self-check rubric

- ☐ Stored menuItems in state with useState and called the setter you declared (setMenu).
- ☐ Rendered the list with map inside a ScrollView , with a key on each ItemCard .
- ☐ Wrote a props type for ItemCard and destructured { item } .
- ☐ Showed the Popular badge conditionally with item.popular && (…) .
- ☐ Wired TextInput (value / onChangeText) and Switch (value / onValueChange) to state.
- ☐ Defined addMenuItem in App and passed it down as onAdd , typed as a function.
- ☐ Called it from an arrow function: onPress={() => onAdd({ name, popular })} .
- ☐ Added immutably with [...menu, item] (or the updater form) and cleared the input.
- ☐ Ran the app often and read at least one error or squiggle before changing code.
- ☐ Talked through your plan out loud, including what you would do next when time ran out.

Mock 2: Habit Tracker

> **In one sentence.** Change one item with `map` plus spread, remove one with `filter`, and compute counts from state instead of storing them.

The handout: Habit Tracker



You are building a daily habit checklist. Tapping a habit marks it done or not done. **Remove** deletes it. A summary line shows how many are done, and an empty list shows a friendly message instead of a blank screen.

Given type

```
type Habit = {
  id: number;
  label: string;
  done: boolean;
};
```

Step-by-step breakdown

1. Show every habit in `startingHabits` as a row, using a `HabitRow` component.
2. Tapping a row toggles its `done` value. Done rows look different (mint background, check mark, struck-through text).
3. **Remove** deletes that habit from the list.
4. Show "3 of 5 done" above the list, and a message when there are no habits.
5. Stretch: add a habit with a text input, or a **Clear done** button.

Parts and time budget

- **Part 1: Render the habits (about 5 min).** Put `startingHabits` into state in `App`. Give `HabitRow` a typed `habit` prop and show its label. Render one `HabitRow` per habit inside a `ScrollView`.
- **Part 2: Toggle done with map + spread (about 7 min).** Write `toggleHabit(id: number)` in `App`. It flips `done` for the habit with that `id` and leaves the others alone. Pass it to `HabitRow` as `onToggle`; tapping the row calls it. Done rows get the mint background and struck-through grey text.
- **Part 3: Remove with filter (about 4 min).** Add a **Remove** button to each row. Pressing it deletes that habit from the list. Use a second callback prop, `onRemove`.

- **Part 4: Derived count and an empty state (about 5 min).** Above the list, show “3 of 5 done”, and keep it correct after toggles and removals. When every habit has been removed, show “No habits yet. Enjoy the free time!” instead of an empty list.
- **Stretch: add a habit, or Clear done (if time).** Pick one. (a) A `TextInput` and **Add** button that append `{ id, label, done: false }`. (b) A **Clear done** button that removes every done habit.

The starter App.tsx is on the Mock Interview page.

Complete final solution · Mock 2: Habit Tracker

One file, ready to paste into Snack. The row has a round check mark, and the toggle and **Remove** live in separate `Pressable`s.

✓ App.tsx · complete solution

```
import { useState } from "react";
import { View, Text, Pressable, ScrollView, StyleSheet } from "react-native";

type Habit = {
  id: number;
  label: string;
  done: boolean;
};

type HabitRowProps = {
  habit: Habit;
  onToggle: (id: number) => void;
  onRemove: (id: number) => void;
};

const startingHabits: Habit[] = [
  { id: 1, label: "Drink water", done: true },
  { id: 2, label: "Read 20 pages", done: false },
  { id: 3, label: "Morning stretch", done: true },
  { id: 4, label: "Walk outside", done: false },
  { id: 5, label: "Review CS notes", done: true },
];

const HabitRow = ({ habit, onToggle, onRemove }: HabitRowProps) => {
  return (
    <View style={[styles.row, habit.done && styles.rowDone]}>
      <Pressable
        style={styles.toggleArea}
        onPress={() => onToggle(habit.id)}
        accessibilityRole="checkbox"
      >
        <View style={[styles.check, habit.done && styles.checkDone]}>
          <Text style={styles.checkMark}>{habit.done ? "✓" : ""}</Text>
        </View>
        <Text style={[styles.label, habit.done && styles.labelDone]}>{habit.label}</Text>
      </Pressable>
      <Pressable onPress={() => onRemove(habit.id)} hitSlop={8}>
        <Text style={styles.remove}>Remove</Text>
      </Pressable>
    </View>
  );
};

export default function App() {
  const [habits, setHabits] = useState(startingHabits); // inferred: Habit[]

  // Part 2: update ONE item -> map + spread (a new array of new/old objects)
  const toggleHabit = (id: number) => {
    setHabits(
      habits.map((habit) =>
```

```
        habit.id === id ? { ...habit, done: !habit.done } : habit
      )
    );
  };

  // Part 3: remove ONE item -> filter keeps everything else
  const removeHabit = (id: number) => {
    setHabits(habits.filter((habit) => habit.id !== id));
  };

  // Part 4: derived value -- computed during render, NOT stored in state
  const doneCount = habits.filter((habit) => habit.done).length;

  const rows = habits.map((habit) => (
    <HabitRow key={habit.id} habit={habit} onToggle={toggleHabit} onRemove={removeHabit} />
  ));

  return (
    <View style={styles.container}>
      <Text style={styles.title}>Habit Tracker</Text>
      <Text style={styles.summary}>
        {doneCount} of {habits.length} done
      </Text>
      <ScrollView style={styles.list}>
        {habits.length === 0 ? (
          <Text style={styles.empty}>No habits yet. Enjoy the free time!</Text>
        ) : (
          rows
        )}
      </ScrollView>
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, paddingTop: 60, paddingHorizontal: 20, backgroundColor: "#f4f6ff" },
  title: { fontSize: 30, fontWeight: "800", color: "#1b1f3b" },
  summary: { fontSize: 16, color: "#5f6690", marginBottom: 14 },
  list: { flex: 1 },
  row: {
    backgroundColor: "#ffffff",
    borderRadius: 14,
    padding: 14,
    marginBottom: 10,
    flexDirection: "row",
    alignItems: "center",
    justifyContent: "space-between",
  },
  rowDone: { backgroundColor: "#e6faf4" },
  toggleArea: { flexDirection: "row", alignItems: "center", gap: 12, flex: 1 },
  check: {
    width: 28,
    height: 28,
    borderRadius: 14,
    borderWidth: 2,
    borderColor: "#b8bfe0",
    alignItems: "center",
    justifyContent: "center",
  },
  checkDone: { backgroundColor: "#00b894", borderColor: "#00b894" },
  checkMark: { color: "#ffffff", fontWeight: "900" },
  label: { fontSize: 17, color: "#1b1f3b" },
  labelDone: { color: "#7a8099", textDecorationLine: "line-through" },
  remove: { color: "#d63031", fontWeight: "700" },
  empty: { fontSize: 16, color: "#5f6690", textAlign: "center", marginTop: 40 },
});
```

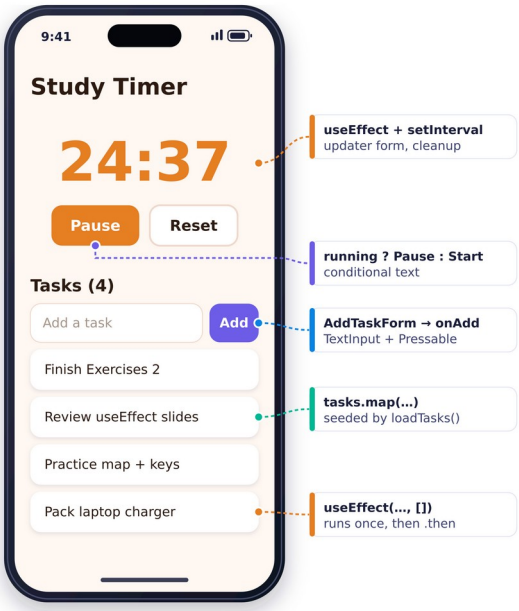
Self-check rubric

- ☐ Stored the habits in state and rendered them with map , with key={habit.id} .
- ☐ Typed HabitRow 's props, including function types for onToggle and onRemove .
- ☐ Toggled with map + { ...habit, done: !habit.done } , without mutating.
- ☐ Removed with filter((habit) => habit.id !== id) .
- ☐ Sent the id up from the child inside an arrow: () => onToggle(habit.id) .
- ☐ Styled done rows with a style array such as [styles.row, habit.done && styles.rowDone] .
- ☐ Computed “X of Y done” from state during render instead of storing it.
- ☐ Showed an empty-state message with a ternary or a real boolean condition.
- ☐ Read an error or squiggle and fixed it yourself.
- ☐ Said your plan out loud before each part.

Mock 3: Study Timer & Tasks

> **In one sentence.** Run a timer in `useEffect` with cleanup and the updater form, load data once with `[]` and `.then`, and add items through a child callback.

The handout: Study Timer & Tasks



You are building a focus timer with a small task list. **Start** counts down from 25:00 once per second, and the button then reads **Pause**. **Reset** goes back to 25:00. Tasks load from a pretend server when the app opens, and you can add more.

Given

```
type Task = {
  id: number;
  title: string;
};
// PROVIDED in the starter:
// loadTasks(): Promise<Task[]>   resolves after 1 second
// formatTime(1500)               returns "25:00"
```

Step-by-step breakdown

1. Add state for the seconds left and whether the timer is running. **Start**/**Pause** toggles it and the label changes. **Reset** works.
2. Make the timer count down once per second while running, using `useEffect` and `setInterval`. Clean up the interval.
3. When the app first renders, call `loadTasks()` and show the tasks. Show "Loading tasks…" until they arrive.
4. Add a task with a `TextInput` and an **Add** button in an `AddTaskForm` child component.
5. Stretch: tap a task to remove it.

Parts and time budget

- **Part 1: Timer state and a Start/Pause button (about 5 min).** Add `secondsLeft` (starting at `25 * 60`) and `running` (starting at `false`) to state. Show `formatTime(secondsLeft)`. Add a button whose label is **Start** when stopped and **Pause** when running; pressing it flips `running`. Add **Reset**, which stops the timer and puts it back to 25:00. Nothing needs to count down yet.

- **Part 2: Count down with useEffect and setInterval (about 7 min).** While `running` is `true`, subtract one from `secondsLeft` every second. When the user pauses, the countdown must stop, and no interval may keep running in the background. Do not go below 0:00.
- **Part 3: Load tasks once with a Promise (about 5 min).** When the app first renders, call the provided `loadTasks()`. When its Promise resolves, put the tasks in state and render them with `map` under a **Tasks** heading. Until then, show "Loading tasks…".
- **Part 4: Add a task from a child form (about 6 min).** Build `AddTaskForm` with a `TextInput` and an **Add** button. It reports the typed title up through an `onAdd(title: string)` prop. `App` creates the new `Task` and appends it. Ignore empty titles, and clear the input after adding.
- **Stretch: tap a task to remove it (if time).** Wrap each task row in a `Pressable`. Tapping it removes that task. (Auto-stop at 0:00 is already in the Part 2 solution.)

The starter App.tsx is on the Mock Interview page.

Complete final solution · Mock 3: Study Timer & Tasks

One file, ready to paste into Snack.

✓ App.tsx · complete solution

```
import { useEffect, useState } from "react";
import { View, Text, TextInput, Pressable, ScrollView, StyleSheet } from "react-native";

type Task = {
  id: number;
  title: string;
};

type AddTaskFormProps = { onAdd: (title: string) => void };

// PROVIDED: pretends to load tasks from a server. Resolves after 1 second.
const loadTasks = (): Promise<Task[]> => {
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve([
        { id: 1, title: "Finish Exercises 2" },
        { id: 2, title: "Review useEffect slides" },
        { id: 3, title: "Practice map + keys" },
      ]);
    }, 1000);
  });
};

// PROVIDED: 1500 -> "25:00"
const formatTime = (totalSeconds: number): string => {
  const minutes = Math.floor(totalSeconds / 60);
  const seconds = totalSeconds % 60;
  return `${minutes}:${seconds < 10 ? "0" : ""}${seconds}`;
};

const START_SECONDS = 25 * 60;

const AddTaskForm = ({ onAdd }: AddTaskFormProps) => {
  const [title, setTitle] = useState("");

  const handleAdd = () => {
    if (title.trim().length === 0) return; // ignore empty titles
    onAdd(title.trim());
    setTitle("");
  };

  return (
```

```
    <View style={styles.form}>
      <TextInput
        style={styles.input}
        value={title}
        onChangeText={setTitle}
        placeholder="Add a task"
        onSubmitEditing={handleAdd}
      />
      <Pressable style={styles.addButton} onPress={handleAdd}>
        <Text style={styles.addButtonText}>Add</Text>
      </Pressable>
    </View>
  );
};

export default function App() {
  // Part 1: timer state
  const [secondsLeft, setSecondsLeft] = useState(START_SECONDS);
  const [running, setRunning] = useState(false);

  // Part 3: task state starts EMPTY, so give useState the type
  const [tasks, setTasks] = useState<Task[]>([]);
  const [loading, setLoading] = useState(true);

  // Part 2: tick once per second while running; clean up the interval
  useEffect(() => {
    if (!running) return;
    const id = setInterval(() => {
      setSecondsLeft((prev) => (prev > 0 ? prev - 1 : 0)); // updater form
    }, 1000);
    return () => clearInterval(id);
  }, [running]);

  // Part 2 (polish): stop automatically at 0:00
  useEffect(() => {
    if (secondsLeft === 0) setRunning(false);
  }, [secondsLeft]);

  // Part 3: load once, after the first render
  useEffect(() => {
    loadTasks().then((loaded) => {
      setTasks(loaded);
      setLoading(false);
    });
  }, []);

  // Part 4: handler lives in the parent, where tasks state lives
  const addTask = (title: string) => {
    setTasks([...tasks, { id: Date.now(), title }]);
  };

  const reset = () => {
    setRunning(false);
    setSecondsLeft(START_SECONDS);
  };

  const taskRows = tasks.map((task) => (
    <View key={task.id} style={styles.task}>
      <Text style={styles.taskText}>{task.title}</Text>
    </View>
  ));

  return (
    <View style={styles.container}>
      <Text style={styles.title}>Study Timer</Text>
      <Text style={styles.clock}>{formatTime(secondsLeft)}</Text>
```

```
      <View style={styles.buttonRow}>
        <Pressable
          style={[styles.button, running && styles.buttonPause]}
          onPress={() => setRunning(!running)}
        >
          <Text style={styles.buttonText}>{running ? "Pause" : "Start"}</Text>
        </Pressable>
        <Pressable style={[styles.button, styles.buttonGhost]} onPress={reset}>
          <Text style={[styles.buttonText, styles.buttonGhostText]}>Reset</Text>
        </Pressable>
      </View>

      <Text style={styles.heading}>Tasks ({tasks.length})</Text>
      <AddTaskForm onAdd={addTask} />
      <ScrollView style={styles.list}>
        {loading ? <Text style={styles.muted}>Loading tasks…</Text> : taskRows}
      </ScrollView>
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, paddingTop: 60, paddingHorizontal: 20, backgroundColor: "#fff7f2" },
  title: { fontSize: 30, fontWeight: "800", color: "#2d1b12" },
  clock: { fontSize: 64, fontWeight: "800", textAlign: "center", color: "#e67e22",
   marginVertical: 20 },
  buttonRow: { flexDirection: "row", justifyContent: "center", gap: 12 },
  button: { backgroundColor: "#00b894", borderRadius: 12, paddingVertical: 12, paddingHorizontal:
   28 },
  buttonPause: { backgroundColor: "#e67e22" },
  buttonGhost: { backgroundColor: "#ffffff", borderWidth: 2, borderColor: "#f0d9cc" },
  buttonText: { color: "#ffffff", fontWeight: "700", fontSize: 17 },
  buttonGhostText: { color: "#2d1b12" },
  heading: { fontSize: 20, fontWeight: "700", color: "#2d1b12", marginTop: 28 },
  form: { flexDirection: "row", gap: 8, marginVertical: 12 },
  input: {
    flex: 1,
    backgroundColor: "#ffffff",
    borderWidth: 1,
    borderColor: "#f0d9cc",
    borderRadius: 12,
    padding: 12,
    fontSize: 16,
  },
  addButton: { backgroundColor: "#6c5ce7", borderRadius: 12, paddingHorizontal: 18,
   justifyContent: "center" },
  addButtonText: { color: "#ffffff", fontWeight: "700" },
  list: { flex: 1 },
  task: { backgroundColor: "#ffffff", borderRadius: 12, padding: 14, marginBottom: 8 },
  taskText: { fontSize: 16, color: "#2d1b12" },
  muted: { fontSize: 16, color: "#8a7468" },
});
```

Self-check rubric

- ☐ Created secondsLeft and running state and showed formatTime(secondsLeft) .
- ☐ Switched the label with a ternary: running ? "Pause" : "Start" .
- ☐ Started setInterval inside useEffect with [running] as the dependency array.
- ☐ Returned a cleanup function that calls clearInterval .
- ☐ Used the updater form in the interval, setSecondsLeft((prev) => prev - 1) , and could explain why.
- ☐ Loaded tasks in useEffect(..., []) with loadTasks().then(...) .
- ☐ Typed the empty array with useState<Task[]>([]) and showed a loading message.
- ☐ Rendered tasks with map and key={task.id} .
- ☐ Passed onAdd down to AddTaskForm and appended immutably in the parent.

- [ ] Read errors carefully and talked through your reasoning.

Rapid-fire questions with model answers

Say each answer out loud in one or two sentences before you read the model answer.

Arrow functions

Q1. What is the difference between `(x) => x * 2` and `(x) => { x * 2 }`?

Answer: The first has an **expression body**, so it returns `x * 2` automatically. The second has a **block body**, because the braces start a function body, and a block body only returns what you `return`. It returns `undefined`. Write `(x) => { return x * 2; }` if you want braces.

Q2. How do you return an object literal from a one-line arrow function?

Answer: Wrap the object in parentheses: `() => ({ name: "Latte", popular: true })`. Without them, JavaScript reads `{` as the start of a function body, not an object. This is why `menu => {...menu, item}` does not do what it looks like.

```
const makeItem = (name: string) => ({ name, popular: false });
```

Q3. What is the difference between `onPress={handleAdd}`, `onPress={handleAdd()}` and `onPress={() => handleAdd(item)}`?

Answer: `onPress={handleAdd}` passes the function itself, to be called on tap. `onPress={() => handleAdd(item)}` passes a new function that calls `handleAdd` with an argument on tap. `onPress={handleAdd()}` **calls it during render** and passes its return value. If it sets state, that causes "Too many re-renders."

Q4. What is an anonymous function, and where do you use them in React Native?

Answer: An anonymous function is a function without a name, usually an arrow written inline, such as `() => setCount(count + 1)`. In React Native you write them constantly: in `onPress`, in `onChangeText`, as the callback passed to `map`, and inside `useEffect` and `setInterval`. Functions are values in JavaScript, so you can pass them around like numbers or strings.

Q5. When must you use parentheses around arrow parameters?

Answer: With exactly one untyped parameter, parentheses are optional: `x => x * 2`. You need them for zero parameters `() =>`, two or more `(a, b) =>`, and whenever you add a type, as in `(item: MenuItem) =>`. Always writing them is a simple habit that avoids the question.

Q6. Why does `onChangeText={setName}` work even though there is no arrow?

Answer: `TextInput` calls `onChangeText` with one argument, the new string. `setName` is a function that takes the new value, so passing it directly is fine. It is equivalent to `onChangeText={(text) => setName(text)}`. Use the arrow version when you need to do anything extra, such as trimming.

map & lists

Q7. What does `map` return, and does it change the original array?

Answer: `map` returns a **new array** of the same length, where each element is whatever your callback returned. It does not change the original. In React you use it to turn an array of data into an array of JSX elements.

```
const names = menu.map((item) => item.name); // string[]
```

Q8. Why does every element returned from `map` need a `key`, and what makes a good key?

Answer: React uses keys to match each item in the new render with the same item in the previous render, so it updates the right one when items are added, removed or reordered. A good key is **unique among siblings and stable**. Use a unique id if the data has one (`key={item.id}`); `key={item.name}` is fine only if names are unique. Avoid `Math.random()`, and avoid the index when items can be inserted or removed.

Q9. Where does the `key` go: on the element returned by `map`, or inside the child component?

Answer: On the outermost element **returned from `map`**, for example `<ItemCard key={item.name} item={item} />`. A key inside `ItemCard` does nothing for the list. `key` is also not passed to the child as a prop, so pass the id separately if the child needs it.

Q10. This renders nothing and shows no error. Why? `menu.map((item) => { <ItemCard item={item} /> })`

Answer: The braces start a function **body**, and there is no `return`, so each call returns `undefined`. React renders `undefined` as nothing. Fix it by dropping the braces, so the expression is returned automatically (the `( )` only group multi-line JSX), or keep the braces and add `return`.

```
menu.map((item) => (
  <ItemCard key={item.name} item={item} />
));
```

Q11. Where do you put the result of `map` in your JSX?

Answer: Store it in a variable and insert it with curly braces inside a container, for example `<ScrollView>{cards}</ScrollView>`. You can also write the `map` directly inside the braces. Either way, React renders the array of elements in order.

Q12. How would you show only the popular items?

Answer: Chain `filter` before `map`: `filter` keeps the items that pass a test, and `map` turns the result into components. Both return new arrays, so the state is untouched.

```
const popularCards = menu
  .filter((item) => item.popular)
  .map((item) => <ItemCard key={item.name} item={item} />);

// a count is derived from state, not stored in more state
const popularCount = menu.filter((m) => m.popular).length;
```

Spread & immutable updates

Q13. Why is `menu.push(item); setMenu(menu);` wrong?

Answer: `push` **mutates** the existing array, and then you pass React the same array back. React compares the old and new values, sees the same array, and may skip the re-render, so the screen does not update. Make a new array instead: `setMenu([...menu, item])`.

Q14. Fix this: `setMenu(menu => {...menu, item})`.

Answer: First, `{` right after `=>` starts a function body, so nothing is returned. Second, `menu` is an array, so the copy uses square brackets. Both of these work: the direct form `setMenu([...menu, item])`, and the updater form `setMenu((prev) => [...prev, item])`. Also make sure you call the setter you declared.

Q15. How do you add an item to the start of an array in state?

Answer: Put the new item before the spread: `setMenu([item, ...menu])`. Spread copies the old elements in order, so whatever you write before or after it decides the position.

Q16. How do you toggle one field on one item in an array of objects?

Answer: Use `map` to build a new array. For the matching item, return a copy with the field overridden. Return every other item unchanged. Later properties in an object literal win, so `done` after `...habit` overrides the old value.

```
setHabits(habits.map((habit) =>
  habit.id === id ? { ...habit, done: !habit.done } : habit
));
// same move on the menu: toggle popular for one item
setMenu(menu.map((m) => (m.name === target ? { ...m, popular: !m.popular } : m)));
```

Q17. How do you remove one item from an array in state?

Answer: Use `filter` and keep every item except the one to remove: `setHabits(habits.filter((habit) => habit.id !== id))`. `filter` returns a new array and leaves the old one alone. Avoid `splice`, which mutates.

```
setMenu(menu.filter((m) => m.name !== target));
```

Q18. What does `{ ...item, popular: true }` produce?

Answer: A **new object** with every property of `item` copied over, and then `popular` set to `true`. Properties written later win. The original `item` is unchanged. Spread is shallow, so nested objects are shared, not copied.

State (useState)

Q19. Why can't you just use a normal `let` variable instead of `useState`?

Answer: Changing a normal variable does not tell React anything, so the screen does not update. Also, a component is a function that runs from the top on each render, so a local variable is reset every time. `useState` keeps the value between renders, and calling its setter asks React to re-render.

Q20. You call `setCount(count + 1)` and then `console.log(count)`. What prints, and why?

Answer: It prints the **old** value. A setter does not change the variable in the code that is already running. It requests a re-render, and the new value appears in the next render. Each render sees its own snapshot of state.

Q21. What are the two ways to call a state setter, and when do you use each?

Answer: The **direct form** passes the next value: `setMenu([...menu, item])`. It is the simple default and fine for most handlers. The **updater form** passes a function that receives the latest value: `setMenu((prev) => [...prev, item])`. It is safer when the next value depends on the previous one and several updates can queue up, or inside timers that would otherwise see an old snapshot.

Q22. How does TypeScript know the type of `menu` in `const [menu, setMenu] = useState(menuItems)`?

Answer: TypeScript **infers** it from the initial value. Since `menuItems` is `MenuItem[]`, `menu` is `MenuItem[]` and `setMenu` only accepts a `MenuItem[]` or an updater that returns one. You only need `useState<MenuItem[]>([])` when the initial value is empty or `null` and says nothing about the type.

Q23. Where should a piece of state live?

Answer: In the **closest common parent** of every component that needs it. If only the form needs the text input value, the form can own it. The menu list is read by `App` (to render cards) and changed by the form (to add), so it lives in `App` and the form gets a callback.

Q24. Can you call `useState` inside an `if` or inside a loop?

Answer: No. Hooks must be called at the **top level** of the component, in the same order on every render, never inside conditions, loops, nested functions, or after an early `return`. React matches each `useState` call to its stored value by call order.

Components & props

Q25. What is a props object, and what does `({ item })` in a component's parameter mean?

Answer: Everything a parent writes as attributes, such as `<ItemCard item={x} />`, arrives in the child as one object, `{ item: x }`. `({ item })` destructures that object, which pulls out `item` as a local variable. It is the same as writing `(props)` and then using `props.item`.

Q26. How do you type a component's props in TypeScript?

Answer: Describe the props object with a type and put it after the destructuring pattern.

```
type ItemCardProps = { item: MenuItem };
const ItemCard = ({ item }: ItemCardProps) => { /* ... */ };
```

Q27. What is the difference between a default export and a named export?

Answer: A file can have **one** default export, imported without braces under any name: `import ItemCard from "./ItemCard"`. A file can have **many** named exports, imported with braces and the exact name: `import { MenuItem } from "./types"`. Mixing them up gives "Element type is invalid ... got: undefined."

Q28. Why must component names start with a capital letter?

Answer: JSX treats lowercase tags as built-in elements and capitalized tags as components. `<itemCard />` would not call your function. Write `ItemCard`.

Callbacks & data flow

Q29. What does "one-way data flow" mean in React?

Answer: Data flows **down** from parent to child through props. A child cannot reach up and change its parent's state, or a sibling's. To let a child cause a change, the parent passes down a **function** that the child calls.

Q30. A child form needs to add an item to a list that lives in `App`. Walk me through it.

Answer: Define the handler in `App`, where the state lives: `addMenuItem = (item: MenuItem) => setMenu([...menu, item])`. Pass it down as a prop: `<AddItemForm onAdd={addMenuItem} />`. In the child, call it from the button inside an arrow function, with an object of the right shape: `onPress={() => onAdd({ name, popular })}`.

Q31. How do you type a callback prop?

Answer: Write a function type: parameter names with types, then `=>`, then the return type, usually `void` for event callbacks.

```
type AddItemFormProps = { onAdd: (item: MenuItem) => void };
```

Q32. Why does the child write `onPress={() => onAdd({ name, popular })}` instead of `onPress={onAdd}`?

Answer: `onAdd` needs a `MenuItem`, but `Pressable` calls its `onPress` with a press event, not a menu item. The arrow lets you build the right object and call `onAdd` with it, only when the user taps. `onPress={onAdd({ name, popular })}` without the arrow would call it during render.

Q33. What does "lifting state up" mean?

Answer: When two components need the same changing data, move the state out of them into their closest common parent. The parent passes the value down as a prop, and passes a callback down so children can request changes. This keeps a **single source of truth**.

Q34. What does `{ name, popular }` mean when the variables `name` and `popular` exist?

Answer: It is shorthand for `{ name: name, popular: popular }`. When a property's name matches the variable holding its value, you can write it once. Here it builds a `MenuItem` from the form's two state values.

Inputs

Q35. What are the key props of a controlled `TextInput`?

Answer: `value` is the text to display, usually a state variable. `onChangeText` is called with the new string whenever the user types, and you pass the setter: `onChangeText={setName}`. `placeholder` shows grey hint text while the field is empty.

Q36. How do you clear a `TextInput` after the user adds an item?

Answer: Set its state back to an empty string, `setName("")`, right after calling `onAdd`. Because the input is controlled, it shows whatever is in state, so it clears on the next render.

Q37. What is the difference between `Button` and `Pressable`?

Answer: `Button` is a simple platform button that takes a `title` string and `onPress`, with very limited styling. `Pressable` is a wrapper that detects presses on any children. You put a `Text` inside and style it however you like. Both use `onPress`.

```
<Button title="Add" onPress={handleAdd} />
<Pressable style={styles.addButton} onPress={handleAdd}>
  <Text style={styles.addButtonText}>Add</Text>
</Pressable>
```

Q38. Which props does a `Switch` use?

Answer: `value` (a boolean from state) and `onValueChange`, which is called with the new boolean. So `<Switch value={popular} onValueChange={setPopular} />` is a complete controlled switch.

Conditional rendering

Q39. Name three ways to render something conditionally.

Answer: (1) An `if`/`else` that assigns JSX to a variable, which you then render with `{content}`. (2) A ternary, `cond ? <A /> : <B />`, for two choices. (3) `cond && <A />` for "A or nothing".

Q40. Why can `{items.length && <List />}` crash on a phone?

Answer: When the list is empty, `items.length` is `0`, and `0 && ...` evaluates to `0`. React renders that `0` as a bare string outside a `<Text>`, which gives "Text strings must be rendered within a <Text> component." Use a real boolean: `items.length > 0 && <List />`.

Q41. How would you show "Loading..." until data arrives?

Answer: Keep `loading` in state, starting at `true`, and set it to `false` when the data arrives. Then render `{loading ? <Text>Loading…</Text> : rows}`. The same ternary gives an empty state: `{menu.length === 0 ? <Text>No items yet</Text> : cards}`.

TypeScript

Q42. When do you need to write the type in `useState<…>`?

Answer: When the initial value does not show the full type, usually an empty array or `null`. `useState([])` would give you an array of `never`. Write `useState<MenuItem[]>([])` instead. When the initial value already has the right type, such as `useState(menuItems)`, let inference do the work.

Q43. What does `?` mean in `type Song = { title: string; note?: string }`?

Answer: `note` is an **optional** property. An object can leave it out, and its type is `string | undefined`, so check it before using it.

Q44. What is the difference between a `type` alias and an `interface` for props?

Answer: Both describe the shape of an object, and for props they are interchangeable. `type` can also name unions, such as `type Size = "S" | "M" | "L"`. Pick one style and stay consistent; the starter code here uses `type`.

Q45. What is a good order of work when TypeScript errors slow you down?

Answer: Get the logic working first, the way you would in plain JavaScript, and let inference type most things. Then add types where TypeScript needs them: props, callback props, and empty `useState` arrays. Read each red squiggle's message; it usually names the expected and actual types. Type errors do not stop a Snack from running.

Layout & styles

Q46. What is the default `flexDirection` in React Native, and how does that differ from the web?

Answer: React Native defaults to `column`, so children stack top to bottom. The web defaults to `row`. To put a name and a badge side by side, set `flexDirection: "row"` on their parent.

Q47. `justifyContent` versus `alignItems`: what does each control?

Answer: `justifyContent` positions children along the **main axis**, the direction of `flexDirection`. `alignItems` positions them along the **cross axis**, perpendicular to it. In a row, `justifyContent: "space-between"` spreads items left to right, and `alignItems: "center"` centers them vertically.

Q48. How do you apply a style only when a condition is true?

Answer: Pass an array of styles. Later entries override earlier ones, and `false`, `null` or `undefined` entries are ignored. So `style={[styles.row, habit.done && styles.rowDone]}` adds `rowDone` only for done habits. `Pressable` can also take a function that receives `pressed`, for a tap effect.

```
style={[styles.card, item.popular && styles.popularCard]}
style={({ pressed }) => [styles.btn, pressed && { opacity: 0.6 }]}  // on a Pressable
```

Q49. What does `flex: 1` do?

Answer: It makes a component grow to fill the available space along its parent's main axis. On the root `View` it fills the screen. On a `ScrollView` above a form, it takes the leftover height so the form stays at the bottom.

Effects & Promises

Q50. What do the three dependency-array options of `useEffect` mean?

Answer: No array: run after **every** render. `[]`: run once, after the **first** render. `[a, b]`: run after the first render and then whenever `a` or `b` changes.

Q51. When does an effect's cleanup function run?

Answer: Right before the effect re-runs because its dependencies changed, and when the component unmounts. That is where you call `clearInterval` or `clearTimeout`, so old timers do not pile up.

```
useEffect(() => {
  const id = setInterval(() => setSeconds((s) => s + 1), 1000);
  return () => clearInterval(id);
}, []);
```

Q52. Why use `setSeconds((s) => s + 1)` instead of `setSeconds(seconds + 1)` inside `setInterval`?

Answer: The interval's callback was created during one render, so it keeps seeing that render's `seconds`. With the direct form it would set the same value on every tick. The updater form receives the latest value from React each time, so it counts correctly.

Q53. How do you use a Promise-returning function such as `loadTasks()` in a component?

Answer: Call it inside `useEffect` with `[]` so it runs once, and use `.then` to put the result in state. Do not make the effect function itself `async`, because an effect must return nothing or a cleanup function.

```
useEffect(() => {
  loadTasks().then((loaded) => setTasks(loaded));
}, []);
```

Mental model

Q54. What does “UI is a function of state” mean?

Answer: Your component is a function that takes props and state and returns what the screen should look like. You never edit the screen directly. You change state, React calls your function once more, and it updates the screen to match the new result.

Q55. What happens, step by step, when the user taps a button that calls `setCount(count + 1)`?

Answer: The `onPress` arrow runs and calls the setter, which queues the new value and asks React to re-render. React re-runs the component function, `useState` returns the new value, and the new JSX is compared with the old JSX. Only what changed is updated on screen.

Q56. If a parent re-renders, what happens to its children?

Answer: By default the children re-render too, because the parent's new JSX includes them with possibly new props. A state change re-renders the component that owns the state and the subtree below it, not the whole app.

Interview craft

Q57. You are stuck on an error you do not understand. What do you do?

Answer: Read the whole message out loud, and note the file and line. Say what you expected and what you see. If it still makes no sense, search the error text or the docs for the concept. That is a resourceful search, not a request for the solution.

Q58. Give an example of a good search and a not-so-good search during the interview.

Answer: Good: “React Native Switch props”, “JavaScript array filter syntax”, “what does justifyContent do”. Not so good: “how to build a café menu app with an add form in React Native”, which asks for the solution itself. Search for **how a tool works**, not for the task.

Q59. Time is almost up and you are mid-part. What do you say?

Answer: Say what works, what you were about to do next, and the exact code you would write. For example: “Next I'd pass `addMenuItem` down as `onAdd` and call it in `onPress` with an arrow, then clear the input.” That shows understanding, even without finished code.

Part 4 · Cheat Sheet

Tricky stuff: wrong → right → why

Wrong	Right	Why
`setMenu(menu => {...menu, item})`	`setMenu([...menu, item])` or `setMenu((prev) => [...prev, item])`	Braces right after `=>` start a function body, not an object. And state is an array, so build `[...]`, not `{...}`.
`menu.map((i) => { <ItemCard item={i} /> })`	`menu.map((i) => (<ItemCard key={i.name} item={i} />))`	A block body without `return` returns `undefined`, so nothing renders. Drop the braces (parens around the JSX are fine) or add `return`.
`onPress={addItem(x)}`	`onPress={() => addItem(x)}`	The first calls `addItem` during render. If it sets state, you get "Too many re-renders".
`menu.push(item); setMenu(menu);`	`setMenu([...menu, item])`	Same array object, so React sees no change and skips the re-render. Never mutate state.
`<TextInput onChange={setName} />`	`<TextInput onChangeText={setName} />`	`onChange` passes an event object. `onChangeText` passes the string.
`menu.map((m) => <ItemCard item={m} />)`	`<ItemCard key={m.name} item={m} />`	Every list item needs a unique, stable `key` so React can track it.
`{menu.length && <List />}`	`{menu.length > 0 && <List />}`	When the length is `0`, React renders `0` as raw text outside `<Text>`.
`useState([])`	`useState<MenuItem[]>([])`	An empty array is inferred as `never[]`, so nothing can be added to it.
`() => { name: "Tea" }`	`() => ({ name: "Tea" })`	Without parens, the braces are a body and the function returns `undefined`.
`setState([...menu, item])`	`setMenu([...menu, item])`	Only the setter you declared exists. A generic `setState` gives "Cannot find name".
`setCount(count + 1); console.log(count);`	`const next = count + 1; setCount(next); console.log(next);`	State is a snapshot. The variable updates on the next render, not on the next line.
`style="color: red"`	`style={{ color: "red" }}` or `style={styles.title}`	Styles are JavaScript objects with camelCase keys, not CSS strings.
`<View>Hello</View>`	`<View><Text>Hello</Text></View>`	Every piece of text must be inside `<Text>`. Same for `//` comments in JSX: write `{/* note */}`.
Expecting children side by side	Add `flexDirection: "row"`	React Native defaults to `"column"`. The web defaults to `"row"`.
`onAdd(name, popular)`	`onAdd({ name, popular })`	The prop type says `onAdd` takes one `MenuItem` object, so build that object.

Error decoder: message → meaning → fix

Message (exact text)	What it means	Fix
`Text strings must be rendered within a <Text> component.`	A raw string, `0`, `""`, or a `//` comment sits directly inside a `<View>`.	Wrap it in `<Text>`. Use `{/* */}` for comments. Use `count > 0 &&`.
`Each child in a list should have a unique "key" prop.`	A `map` result has no `key`, or two keys repeat.	Add `key={item.id}` (or `key={item.name}` if names are unique) to the outer element returned by `map`.
`Too many re-renders. React limits the number of renders to prevent an infinite loop.`	State is set during render, usually `onPress={setCount(count + 1)}`.	Wrap it: `onPress={() => setCount(count + 1)}`.
`Property 'nmae' does not exist on type 'MenuItem'.`	A typo, or a field missing from your type. With `on type 'never'`, your state came from `useState([])`.	Fix the spelling, add the field to the type, or use `useState<MenuItem[]>([])`.

Message (exact text)	What it means	Fix
`Type 'string' is not assignable to type 'number'.`	You gave a string where a number was expected, such as `price="5"`.	Pass the right type: `price={5}`, or `Number(text)`.
`Type 'MenuItem' is not assignable to type 'never'.` · `Argument of type 'MenuItem' is not assignable to parameter of type 'never'.`	The array state was created with `useState([])`, so it is `never[]`. With the updater form it may also mention `SetStateAction<never[]>`.	`useState<MenuItem[]>([])`.
`Property 'onAdd' is missing in type '{ item: MenuItem; }' but required in type 'RowProps'.`	You rendered a component without a required prop.	Pass the prop, or mark it optional with `onAdd?:`.
`Parameter 'item' implicitly has an 'any' type.`	A standalone function parameter has no type.	`(item: MenuItem) => ...`
`View is not defined` · `Property 'View' doesn't exist` · `Can't find variable: View` · `Cannot find name 'View'.`	The name was never imported or declared, or it is misspelled.	Add it to the import from `"react-native"` or `"react"`. Check capitals: `TextInput`, not `Textinput`.
`Objects are not valid as a React child (found: object with keys {name, popular}).`	You put a whole object in JSX, such as `<Text>{item}</Text>`.	Render one field: `{item.name}`.
`Cannot read property 'map' of undefined` · `menu.map is not a function`	The value is not an array, often after `setMenu(menu.push(x))` made it a number.	Start with an array and update with `[...menu, x]`.
`Adjacent JSX elements must be wrapped in an enclosing tag.`	A component returns two siblings with no parent.	Wrap them in `<View>` or `<>...</>`.
`Maximum update depth exceeded.`	An effect sets state on every render, usually with no dependency array.	Add `[]` or `[dep]`, and only set state when needed.

Golden patterns (quick reference)

> **In one sentence.** If you can type these five from memory, you can build almost any Interview 2 app: **state from data, map to components, add immutably, callback down and up, controlled input.**

1 · State from data

```
// in App: inference gives menu: MenuItem[]
const [menu, setMenu] = useState(menuItems);
// an empty array needs a type:
const [list, setList] = useState<MenuItem[]>([]);
// name + popular state lives in the form (pattern 4)
```

2 · Map data to components

```
// no { after =>, so the JSX is returned
const cards = menu.map((item) => (
  <ItemCard key={item.name} item={item} />
));
// then inside JSX:
<ScrollView>{cards}</ScrollView>
```

2b · The card: typed, destructured props

```
const ItemCard = ({ item }: ItemCardProps) => (
  <View style={styles.card}>
    <Text>{item.name}</Text>
    {item.popular && <Text>Popular</Text>}
  </View>
);
// item.popular is a boolean, so && is safe
```

3 · Add to an array immutably

```
const addMenuItem = (item: MenuItem) => {
  setMenu([...menu, item]); // new array!
  // also valid: setMenu((prev) => [...prev, item]);
};
```

4a · Parent (App): owns the list

```
const addMenuItem = (item: MenuItem) => {
  setMenu([...menu, item]);
};
// pass the function down, no ()
<AddItemForm onAdd={addMenuItem} />
```

4b · Child: owns the text field and switch

```
const AddItemForm = ({ onAdd }: AddItemFormProps) => {
  const [name, setName] = useState("");
  const [popular, setPopular] = useState(false);
  const handleAdd = () => {
    if (name.trim() === "") return;
    onAdd({ name, popular }); // call it UP
    setName("");
    setPopular(false);
  };
  return (
    <View>
      <TextInput value={name} onChangeText={setName} placeholder="Item name" />
      <Switch value={popular} onValueChange={setPopular} />
      <Pressable onPress={handleAdd}><Text>Add</Text></Pressable>
    </View>
  );
};
```

5 · Controlled inputs

```
// value comes from state; the handler calls the setter
<TextInput value={name} onChangeText={setName} />
// same thing: onChangeText={(text) => setName(text)}
<Switch value={popular} onValueChange={setPopular} />
```

App skeleton: type it from memory

> **In one sentence.** This is the 2-minute scaffold: **imports, type, child with typed props, App with state, map, callback, StyleSheet.** Practice typing it until it feels automatic.

```
import { useState } from "react";
import {
  View,
  Text,
  TextInput,
  Pressable,
  ScrollView,
  Switch,
  StyleSheet,
} from "react-native";

// 1. Type + starting data
type MenuItem = {
  name: string;
  popular: boolean;
};

const menuItems: MenuItem[] = [
  { name: "Tea", popular: true },
```

```
  { name: "Toast", popular: false },
];

// 2. Child that displays one item
type ItemCardProps = { item: MenuItem };

const ItemCard = ({ item }: ItemCardProps) => {
  return (
    <View style={styles.card}>
      <Text>{item.name}</Text>
      {item.popular && <Text>Popular</Text>}
    </View>
  );
};

// 3. Child that reports a new item UP
type AddItemFormProps = {
  onAdd: (item: MenuItem) => void;
};

const AddItemForm = ({ onAdd }: AddItemFormProps) => {
  const [name, setName] = useState("");
  const [popular, setPopular] = useState(false);

  const handleAdd = () => {
    if (name.trim() === "") return;
    onAdd({ name, popular });
    setName("");
    setPopular(false);
  };

  return (
    <View style={styles.form}>
      <TextInput
        style={styles.input}
        value={name}
        onChangeText={setName}
        placeholder="Item name"
      />
      <View style={styles.row}>
        <Text>Popular?</Text>
        <Switch value={popular} onValueChange={setPopular} />
      </View>
      <Pressable style={styles.button} onPress={handleAdd}>
        <Text style={styles.buttonText}>Add</Text>
      </Pressable>
    </View>
  );
};

// 4. App owns the state + the callback
export default function App() {
  const [menu, setMenu] = useState(menuItems);

  const addMenuItem = (item: MenuItem) => {
    setMenu([...menu, item]);
  };

  const cards = menu.map((item) => (
    <ItemCard key={item.name} item={item} />
  ));

  return (
    <View style={styles.container}>
      <AddItemForm onAdd={addMenuItem} />
      <ScrollView>{cards}</ScrollView>
    </View>
```

```
  );
}

// 5. Styles
const styles = StyleSheet.create({
  container: { flex: 1, padding: 20, paddingTop: 60, gap: 12 },
  form: { gap: 8 },
  row: {
    flexDirection: "row",
    justifyContent: "space-between",
    alignItems: "center",
  },
  input: { borderWidth: 1, borderColor: "#ccc", borderRadius: 8, padding: 10 },
  button: {
    backgroundColor: "#6c5ce7",
    padding: 12,
    borderRadius: 8,
    alignItems: "center",
  },
  buttonText: { color: "white", fontWeight: "bold" },
  card: {
    padding: 12,
    marginBottom: 8,
    borderRadius: 8,
    backgroundColor: "#f0eeff",
  },
});
```

- Build in this order: type and data, static JSX, state and `map`, then the form and callback.
- Run it after every step so each error points at a few new lines.

Interview-day checklist

> **In one sentence.** They are watching how you think and debug, not whether you finish, so talk aloud, read the errors, and search for knowledge rather than answers.

The format

- ☐ **25 minutes** on one programming problem, in **Expo Snack** on your laptop.
- ☐ You get a **handout** with a reference screenshot and a step-by-step breakdown.
- ☐ You can **ask clarifying questions** at any point. The TA may also ask you to move on to another part.
- ☐ The focus is your **process, not the final code**. You are **not expected to finish** the whole problem.
- ☐ Pencil and paper are provided if you want to sketch the layout first.

Allowed resources

- ☐ The open internet, including the Google Search AI overview.
- ☐ One page of handwritten notes in English, front and back, pen and paper only. You turn it in at the end. (You do not need notes to do well.)

Not allowed

- ☐ Any other AI tools, such as chatbots or code assistants.
- ☐ Homework solutions, including solution-like code written by hand on your notes page.

Before and during

- ☐ **Arrive 5 minutes early** with a **charged laptop**. Delays come out of your 25 minutes.
- ☐ **Think aloud.** Saying "I expect this to show three cards" helps the TA see where you are and point things out.
- ☐ **Read error messages** slowly. They often name the exact problem, like `Text strings must be rendered within a <Text> component`.
- ☐ **Write it in JavaScript first**, run it often, and add TypeScript types once it works.
- ☐ **If the interviewer says move on**, stub the unfinished part with hard-coded values (for example, a fixed array instead of state) and come back to it later.

A 60-second talk-aloud script

- ☐ "Let me read the whole handout first and sketch the screen."
- ☐ "The data is a list of items, so I'll write a type with `name` and `popular`."
- ☐ "The list lives in `App`, because the form needs to add to it and the cards need to show it."
- ☐ "I'll get static cards on screen first, then `map` over state, then the form and the callback."
- ☐ "I'll run it after each step. I expect three cards now… I see three, good."

Snack tips

- ☐ **Red squiggles usually do not stop the app.** TypeScript errors are warnings in Snack, which is why "JavaScript first, types second" works.
- ☐ **The web preview is not a phone.** Text outside `<Text>` or a stray `0` may only crash on iOS or Android.
- ☐ **On the red error screen**, read the message and the line number before changing anything.
- ☐ `onChangeText={setName}` is fine because it passes a string. `onPress={setX}` is not the same: it passes an event object, so write `onPress={() => setX(true)}`.

Good searches: general knowledge

- *what values work for justifyContent?* (Looking up documentation.)

- *what is the syntax for a self-closing tag in React Native?* (Debugging syntax.)
- *what's the difference between justifyContent and alignItems* (Reviewing a concept.)

Searches to avoid: asking for the answer

- *how do I center the links in a navbar?* (Asking for the solution directly.)
- *"this looks like a snippet on my notes, I'll copy it and tweak it"* (Pattern matching instead of reasoning.)

If you get stuck

Getting stuck is a normal part of the interview, and handling it calmly is exactly the process they want to see. More in Interview craft.

Answer Key

Drill answers first, then the full solutions to the exercises in Part 2, then the final Build A file.

Drill answers

D1. A local variable vs. state

```
count: 0, score: 3
```

Why. Each press sets `count` to 1, but only inside that render's copy of the variable. `setScore(score + 1)` stores the new score **in React** and asks for a re-render, so `App` reruns from the top and `let count = 0` makes a brand-new variable at 0. `score` survives because React keeps it between renders: 1, 2, 3. If you deleted the `setScore` line, the screen would never change at all, because assigning to `count` tells React nothing. A variable declared outside the component keeps its value, but it still never triggers a render. Rule: anything that should change what's on screen lives in `useState` and changes through its setter.

D2. One-liner map

```
[2, 4, 8]
```

Why. `map` calls the arrow once per item and collects the return values into a **new array**. The arrow has an expression body (no braces), so `x * 2` is returned automatically. The original `[1, 2, 4]` is unchanged.

D3. Braces but no return

```
[undefined, undefined, undefined]
```

Why. Braces right after `=>` start a **block body**. A block body only returns what you explicitly `return`, so each call returns `undefined`. `map` still produces one item per input, which is why you get three `undefined`s. In JSX this is the "my list renders nothing and there's no error" bug.

D4. Spread makes a copy (Lecture 4b)

```
[1, 2, 4, 8, 16]
[1, 2, 4, 8, 16, 32]
```

Why. `[...array, 32]` builds a **brand-new array**: it copies every element of `array`, then adds 32. The original is untouched. This is the immutable update React wants, because the new array is a different reference, so React can tell that state changed.

D5. Same array, two names

```
[1, 2, 4, 8]
true
```

Why. `const array2 = array` does **not** copy; both names point to the same array. `push` changes that one array, so `array` shows 8 too. This is why `menu.push(item); setMenu(menu)` fails to re-render: the "new" state is the same reference as the old, so React sees no change.

D6. Three setter calls in one press

```
count becomes 1 (not 3)
```

Why. During this press, `count` is a **snapshot** from the current render: it is 0 in all three lines, so each call says "set to 1." To add 3, use the updater form, `setCount((c) => c + 1)`, three times; React queues them and applies them in order.

D7. Adding an item to an array in state

✓ Fixed

```
const addMenuItem = (item: MenuItem) => {
  setMenu([...menu, item]);
  // also valid (updater form): setMenu((prev) => [...prev, item]);
};
```

Why. Two separate problems. First, `{` right after `=>` starts a **function body**, and a body cannot begin with `...`, so this is a syntax error. Second, `menu` is an **array**, so the copy must use square brackets: `[...menu, item]`. Curly braces would build an object. The direct form is the simplest default; the updater form is also correct and is safer when several updates happen at once.

D8. The list renders nothing

✓ Fixed

```
const cards = menu.map((item) => (
  <ItemCard key={item.name} item={item} />
));
// or: menu.map((item) => { return <ItemCard key={item.name} item={item} />; })
```

Why. Braces after `=>` make a block body, which returns `undefined` unless you write `return`. Drop the braces so nothing but the JSX follows `=>`; then it is **returned automatically** (the `( )` around it are just grouping for multi-line JSX). Or keep the braces and add `return`. While you are there, add the `key`.

D9. The app freezes with a render error

✓ Fixed

```
<Pressable onPress={() => addItem(item)}>
  <Text>Add</Text>
</Pressable>
```

Why. `addItem(item)` **calls** the function during render and passes its return value (`undefined`) to `onPress`. If `addItem` sets state, that triggers a render, which calls it once more: **Too many re-renders. React limits the number of renders to prevent an infinite loop.** Wrap it in an arrow so it runs only on press.

D10. The state changes but the screen doesn't

✓ Fixed

```
const addMenuItem = (item: MenuItem) => {
  setMenu([...menu, item]);
};
```

Why. `push` **mutates** the existing array. Passing the same array back to `setMenu` gives React the same reference, and React skips re-rendering when the new value is identical to the old one. Build a new array with spread instead. (And never write `setMenu(menu.push(item))`: `push` returns the new length, a number.)

D11. The name becomes an object

✓ Fixed

```
<TextInput value={name} onChangeText={setName} placeholder="Item name" />
// equivalent: onChangeText={(text) => setName(text)}
```

Why. `onChange` calls your function with an **event object**, so `name` would stop being a string. `onChangeText` calls it with just the new **text**, which is exactly what `setName` expects. TypeScript flags the original because an event is not assignable to a string.

D12. The yellow warning about keys

✓ Fixed

```
{menu.map((item) => (
  <ItemCard key={item.name} item={item} />
))}
```

> **Why.** React needs a **key** on each element returned from `map` so it can match items between renders. Without one you get **Each child in a list should have a unique "key" prop.** Use a unique id if the data has one (`key={item.id}`). `key={item.name}` is fine only if names are unique. Never `Math.random()`, which changes every render.

D13. Works with items, errors at zero

✓ Fixed

```
{count > 0 && <Text>{count} items in cart</Text>}
// or: {count > 0 ? <Text>{count} items in cart</Text> : null}
```

> **Why.** `&&` returns its left side when that side is falsy. With `count` equal to 0, the expression is the number 0, which React Native tries to draw outside a `<Text>`: **Text strings must be rendered within a <Text> component.** Make the left side a real boolean.

D14. Not assignable to type 'never'

✓ Fixed

```
const [menu, setMenu] = useState<MenuItem[]>([]);
```

> **Why.** From an empty `[]`, TypeScript can't guess what will go in the array, so in strict mode it infers `never[]`: an array that must stay empty. Adding a `MenuItem` then fails. Give the element type with a **type argument**: `useState<MenuItem[]>([])`. If you start from data (`useState(menuItems)`), inference handles it.

D15. An arrow that should return an object

✓ Fixed

```
const makeItem = () => ({ name: "Tea", popular: false });
```

> **Why.** The braces are read as a **function body**, not an object. With two properties this is a syntax error; with one property (`() => { name: "Tea" }`) it is worse, because it runs and silently returns `undefined` (`name:` is treated as a label). Wrapping the object in **parentheses** makes it an expression that is returned.

D16. A setter that doesn't exist

✓ Fixed

```
<Pressable onPress={() => setCount(count + 1)}>
  <Text>+1</Text>
</Pressable>
```

> **Why.** `setState` is a placeholder name from generic examples, not a real function in this component. You get **Cannot find name 'setState'** from TypeScript and a ReferenceError when pressed. Call the setter **you declared** in the array destructuring: `setCount`.

D17. Map to cards

Answers: (1) `map` (2) `key`

> **Why.** `map` turns each item into one element; `key` gives each element a stable identity among its siblings.

D18. Add without mutating

Answers: (1) `...`

> **Why.** The spread `...` copies the old items into a new array literal, then `item` is added at the end. New array, new reference, re-render.

D19. A controlled TextInput and Switch

Answers: (1) `onChangeText` (2) `onValueChange`

> **Why.** `onChangeText` passes the new string, `onValueChange` passes the new boolean, and each matches its setter's parameter.

D20. Child calls the parent's callback

Answers: (1) `() =>` (2) `popular`

> **Why.** `() =>` makes a function that runs later, on press. Inside it, the child builds the full object the `MenuItem` type expects: both `name` and `popular`.

D21. Toggle one field

Answers: (1) `...` (2) `!item.popular`

> **Why.** Copy the item with object spread, then override one field; later properties win. Every other item is returned unchanged.

D22. Types for an empty list and a callback prop

Answers: (1) `MenuItem[]` (2) `void`

> **Why.** An empty array needs its element type. A handler that only causes updates returns nothing, which TypeScript calls `void`.

Exercise solutions

Fruit cart: pass a function, don't call it

✓ Solution · App.tsx

```
import { useState } from "react";
import { View, Text, Pressable, StyleSheet } from "react-native";

type FruitData = { price: number; emoji: string };

const fruitList: FruitData[] = [
  { price: 2, emoji: "▫" },
  { price: 3, emoji: "▫" },
  { price: 5, emoji: "▫" },
  { price: 8, emoji: "▫" },
];

export default function App() {
  const [total, setTotal] = useState(0);

  const addToCart = (price: number) => {
    setTotal(total + price);
  };

  // One-liner map: no { after =>, so the JSX is returned automatically
  const buttons = fruitList.map((fruit) => (
    <Pressable
      key={fruit.emoji}
      style={styles.fruit}
      // A function VALUE: it runs only when the user presses
      onPress={() => addToCart(fruit.price)}
```

```
    >
      <Text style={styles.emoji}>{fruit.emoji}</Text>
      <Text style={styles.price}>${fruit.price}</Text>
    </Pressable>
  ));

  return (
    <View style={styles.container}>
      <Text style={styles.total}>Total: ${total}</Text>
      <View style={styles.grid}>{buttons}</View>
      <Pressable style={styles.clear} onPress={() => setTotal(0)}>
        <Text style={styles.clearText}>Clear</Text>
      </Pressable>
    </View>
  );
}

// const styles = StyleSheet.create({ ... }) is unchanged
// from the starter file on the Practice page.
```

Why this works. There are three things you can put in `onPress`: `f` (pass the function itself), `f()` (call it now and pass its result), and `() => f(x)` (pass a new function that will call `f(x)` later). You need the third one whenever the handler needs an argument, like the price of *this* fruit. Because each arrow is created inside `map`, it remembers its own `fruit`.

Common mistakes

- `onPress={addToCart(fruit.price)}` calls `addToCart` during render. That sets state, which renders, which calls it, and so on: **Too many re-renders. React limits the number of renders to prevent an infinite loop.**
- `onPress={addToCart}` passes the function, but React Native calls it with a press event, not a price. TypeScript flags this because the event is not a `number`.
- Forgetting `key={fruit.emoji}` on the Pressable that `map` returns.

Playlist cards with a block-body map

✓ Solution · App.tsx

```
import { View, Text, ScrollView, StyleSheet } from "react-native";

type Song = { title: string; artist: string; liked: boolean };

const songs: Song[] = [
  { title: "Sunrise Drive", artist: "The Maple Lights", liked: true },
  { title: "Paper Boats", artist: "Nova Lane", liked: false },
  { title: "City of Kites", artist: "Juniper Ray", liked: true },
];

type SongCardProps = { song: Song; position: number };

const SongCard = ({ song, position }: SongCardProps) => {
  return (
    <View style={styles.card}>
      <View style={styles.cardText}>
        <Text style={styles.songTitle}>
          {position}. {song.title}
        </Text>
        <Text style={styles.artist}>{song.artist}</Text>
      </View>
      {/* liked is a boolean, so && renders the badge or nothing */}
      {song.liked && (
        <View style={styles.badge}>
          <Text style={styles.badgeText}>Liked</Text>
        </View>
      )}
    </View>
```

```
  );
};

export default function App() {
  // Block body: braces let us run statements first, then we MUST return
  const cards = songs.map((song, index) => {
    const position = index + 1;
    return <SongCard key={song.title} song={song} position={position} />;
  });

  return (
    <View style={styles.container}>
      <Text style={styles.title}>Road trip mix</Text>
      <ScrollView>{cards}</ScrollView>
    </View>
  );
}

// const styles = StyleSheet.create({ ... }) is unchanged
// from the starter file on the Practice page.
```

Why this works. Use the one-liner when the callback only returns JSX. Switch to a block body when you want a variable or an `if` first, and remember that braces turn off the automatic return. The `key` goes on `<SongCard>` (the element `map` returns), not on the `View` inside `SongCard`. `key` is not passed to the component as a prop.

Common mistakes

- Forgetting `return` in the block body. The ScrollView renders empty with no error at all, which is why this bug is so sneaky.
- Putting `key` on the inner `<View style={styles.card}>`. React still warns: **Each child in a list should have a unique "key" prop.**
- Writing the badge text directly inside a `View` without `<Text>`. In React Native every string must be inside `<Text>`.

Four immutable recipes: add, prepend, remove, toggle

✓ Solution · App.tsx

```
import { View, Text, StyleSheet } from "react-native";

type Habit = { id: number; label: string; done: boolean };

const habits: Habit[] = [
  { id: 1, label: "Stretch", done: false },
  { id: 2, label: "Read", done: false },
  { id: 3, label: "Water", done: true },
];

const newHabit: Habit = { id: 4, label: "Walk", done: false };

// Add to the end: copy everything, then the new item
const added: Habit[] = [...habits, newHabit];
// Add to the start: new item first, then copy everything
const prepended: Habit[] = [newHabit, ...habits];
// Remove: keep every habit whose id is NOT 3
const removed: Habit[] = habits.filter((habit) => habit.id !== 3);
// Toggle: copy the matching object with one field changed; reuse the rest
const toggled: Habit[] = habits.map((habit) =>
  habit.id === 2 ? { ...habit, done: !habit.done } : habit
);

const show = (list: Habit[]) =>
  list.map((h) => h.id + ":" + h.label + (h.done ? " (done)" : "")).join(", ");

type RowProps = { label: string; list: Habit[] };
```

```
const Row = ({ label, list }: RowProps) => (
  <View style={styles.row}>
    <Text style={styles.label}>{label}</Text>
    <Text style={styles.value}>{show(list)}</Text>
  </View>
);

export default function App() {
  return (
    <View style={styles.container}>
      <Row label="original (must not change)" list={habits} />
      <Row label="added" list={added} />
      <Row label="prepended" list={prepended} />
      <Row label="removed" list={removed} />
      <Row label="toggled" list={toggled} />
    </View>
  );
}

// const styles = StyleSheet.create({ ... }) is unchanged
// from the starter file on the Practice page.
```

Why this works. React decides whether to re-render by checking if the state value is a **different reference** from before. Each recipe returns a brand-new array, so React sees a change. Inside the toggle, `{ ...habit, done: !habit.done }` copies the object's fields and then overrides one; later properties win. The original row on screen proves nothing was mutated.

Common mistakes

- Using `{ ...habits, newHabit }` (braces) for an array. That builds an object with keys `0`, `1`, `2`, and `newHabit`, not an array.
- Writing `habit.done = !habit.done` inside `map`. That mutates the original object, so the "original" row changes too.
- Using `habits.push(newHabit)`. It changes `habits` and returns the new length (a number), not an array.

Snapshots and the updater form

✓ Solution · App.tsx

```
import { useState } from "react";
import { View, Text, Pressable, StyleSheet } from "react-native";

export default function App() {
  const [chapters, setChapters] = useState(0);
  const [log, setLog] = useState("");

  const readOne = () => {
    // Updater form: React passes in the latest value (including queued updates)
    setChapters((c) => c + 1);
  };

  const readThree = () => {
    // Three queued updaters: c => c + 1, three times, so +3
    readOne();
    readOne();
    readOne();
  };

  const setAndLog = () => {
    setChapters(chapters + 1);
    // Shows the OLD value: `chapters` is this render's snapshot.
    // The new value only exists in the NEXT render.
    setLog("Right after setting: " + chapters);
  };
```

```
  return (
    <View style={styles.container}>
      <Text style={styles.title}>Book club</Text>
      <Text style={styles.big}>{chapters}</Text>
      <Text style={styles.sub}>chapters read</Text>
      <View style={styles.row}>
        <Pressable style={styles.button} onPress={readOne}>
          <Text style={styles.buttonText}>+1</Text>
        </Pressable>
        <Pressable style={styles.button} onPress={readThree}>
          <Text style={styles.buttonText}>+3</Text>
        </Pressable>
      </View>
      <Pressable style={styles.wide} onPress={setAndLog}>
        <Text style={styles.buttonText}>Set and log</Text>
      </Pressable>
      <Text style={styles.sub}>{log}</Text>
    </View>
  );
}

// const styles = StyleSheet.create({ ... }) is unchanged
// from the starter file on the Practice page.
```

Why this works. A setter accepts either a **next value** (`setChapters(chapters + 1)`) or an **updater function** (`setChapters((c) => c + 1)`). Both are correct. The direct form is the simplest default. The updater form is safer when the new value depends on the old one and several updates can happen before the next render, as here or inside a timer. React batches updates during an event and applies the queued updaters in order.

Common mistakes

- Forgetting that the updater form exists. A setter takes either a new value or a function `prev => newValue`; both are part of the `useState` API.
- Calling `setChapters(chapters + 3)` to "fix" it. That works for this button, but it hides the real lesson about snapshots.
- Putting side effects (like `setLog`) *inside* an updater function. Updaters should only compute and return the next value.

Extract a PlantCard component

✓ Solution · App.tsx

```
import { View, Text, ScrollView, StyleSheet } from "react-native";

type Plant = { name: string; waterEveryDays: number; thirsty: boolean };

const plants: Plant[] = [
  { name: "Fern", waterEveryDays: 3, thirsty: true },
  { name: "Cactus", waterEveryDays: 14, thirsty: false },
  { name: "Basil", waterEveryDays: 2, thirsty: true },
];

// Props are ONE object; this type describes its shape
type PlantCardProps = { plant: Plant };

// Destructure the props object right in the parameter list
const PlantCard = ({ plant }: PlantCardProps) => (
  <View style={styles.card}>
    <Text style={styles.name}>{plant.name}</Text>
    <Text style={styles.meta}>Every {plant.waterEveryDays} days</Text>
    <Text style={plant.thirsty ? styles.thirsty : styles.happy}>
      {plant.thirsty ? "Needs water" : "Happy"}
    </Text>
```

```
  </View>
);

export default function App() {
  return (
    <View style={styles.container}>
      <Text style={styles.title}>Plant care</Text>
      <ScrollView>
        {plants.map((plant) => (
          // key stays on the element that map returns
          <PlantCard key={plant.name} plant={plant} />
        ))}
      </ScrollView>
    </View>
  );
}

// const styles = StyleSheet.create({ ... }) is unchanged
// from the starter file on the Practice page.
```

Why this works. When you write `<PlantCard plant={plant} />`, React calls `PlantCard({ plant: plant })`. The braces in `({ plant })` **destructure** that props object so you can write `plant.name` instead of `props.plant.name`. The props type lets TypeScript check every place you use `PlantCard`.

Common mistakes

- Writing `const PlantCard = (plant: Plant) => ...`. The parameter is the whole props object, so `plant.name` is undefined. Destructure: `({ plant }: PlantCardProps)`.
- Naming the component `plantCard`. Lowercase JSX tags are treated as built-in elements, not your component.
- Defining `PlantCard` inside `App`. It works, but it is recreated on every render; define components at the top level of the file.

Playlist: an AddSongForm that calls onAdd

✓ Solution · App.tsx

```
import { useState } from "react";
import { View, Text, TextInput, Pressable, ScrollView, StyleSheet } from "react-native";

type Song = { title: string; artist: string; liked: boolean };

const startingSongs: Song[] = [
  { title: "Sunrise Drive", artist: "The Maple Lights", liked: true },
  { title: "Paper Boats", artist: "Nova Lane", liked: false },
];

type AddSongFormProps = { onAdd: (song: Song) => void };

const AddSongForm = ({ onAdd }: AddSongFormProps) => {
  // The form's own inputs can be local state: only the form needs them
  const [title, setTitle] = useState("");
  const [artist, setArtist] = useState("");

  const handleAdd = () => {
    // Build the full object the Song type expects ({ title, artist } shorthand)
    onAdd({ title, artist, liked: false });
    // Clear the inputs so the next song starts fresh
    setTitle("");
    setArtist("");
  };

  return (
    <View style={styles.form}>
```

```
      <TextInput style={styles.input} value={title} onChangeText={setTitle} placeholder="Song
   title" />
      <TextInput style={styles.input} value={artist} onChangeText={setArtist}
   placeholder="Artist" />
      <Pressable style={styles.button} onPress={handleAdd}>
        <Text style={styles.buttonText}>Add song</Text>
      </Pressable>
    </View>
  );
};

export default function App() {
  const [songs, setSongs] = useState(startingSongs);

  // The handler lives with the state; it gets a complete Song from the child
  const addSong = (song: Song) => {
    setSongs([...songs, song]);
  };

  return (
    <View style={styles.container}>
      <Text style={styles.title}>Playlist</Text>
      <ScrollView style={styles.list}>
        {songs.map((song) => (
          <View key={song.title} style={styles.card}>
            <Text style={styles.songTitle}>{song.title}</Text>
            <Text style={styles.artist}>{song.artist}</Text>
          </View>
        ))}
      </ScrollView>
      <AddSongForm onAdd={addSong} />
    </View>
  );
}

// const styles = StyleSheet.create({ ... }) is unchanged
// from the starter file on the Practice page.
```

Why this works. This is the whole interview pattern in one screen. The **parent** owns the list and the function that changes it. The **child** owns its temporary input text, and when the user presses Add it builds a complete `Song` and hands it up through `onAdd`. If you don't need to clear inputs, the short version is `onPress={() => onAdd({ title, artist, liked: false })}`. A named `handleAdd` is cleaner once there are several steps.

Common mistakes

- Calling `onAdd({ title, artist })` without `liked`. TypeScript reports **Property 'liked' is missing** in that object but required in type 'Song'.
- Writing `onPress={onAdd({ title, artist, liked: false })}` without the arrow. It runs during render and adds an empty song over and over.
- Defining `addSong` inside `AddSongForm` and trying to call `setSongs` there. The child can't see the parent's setter unless it is passed down.

Habit tracker: clear the input after adding, disable Add when empty

✓ Solution · App.tsx

```
import { useState } from "react";
import { View, Text, TextInput, Pressable, ScrollView, StyleSheet } from "react-native";

type Habit = { id: number; label: string; done: boolean };

const startingHabits: Habit[] = [
  { id: 1, label: "Stretch", done: false },
  { id: 2, label: "Read 10 pages", done: false },
```

```
];

export default function App() {
  const [habits, setHabits] = useState(startingHabits);
  const [label, setLabel] = useState("");

  // Derived: recomputed on every keystroke
  const canAdd = label.trim().length > 0;

  const addHabit = () => {
    if (!canAdd) return; // guard, because onSubmitEditing can also call this
    const newHabit: Habit = { id: Date.now(), label: label.trim(), done: false };
    setHabits([...habits, newHabit]);
    setLabel(""); // controlled input: clearing state clears the field
  };

  return (
    <View style={styles.container}>
      <Text style={styles.title}>Habits</Text>
      <ScrollView style={styles.list}>
        {habits.map((habit) => (
          <View key={habit.id} style={styles.row}>
            <Text style={styles.label}>{habit.label}</Text>
          </View>
        ))}
      </ScrollView>
      <View style={styles.formRow}>
        <TextInput
          style={styles.input}
          value={label}
          onChangeText={setLabel}
          placeholder="New habit"
          onSubmitEditing={addHabit}
        />
        <Pressable
          // false entries in a style array are ignored
          style={[styles.button, !canAdd && styles.buttonDisabled]}
          disabled={!canAdd}
          onPress={addHabit}
        >
          <Text style={styles.buttonText}>Add</Text>
        </Pressable>
      </View>
    </View>
  );
}

// const styles = StyleSheet.create({ ... }) is unchanged
// from the starter file on the Practice page.
```

Why this works. Because the TextInput is controlled by `label`, the field shows whatever `label` is. Setting it to `""` empties the field; there is no separate "clear" call to remember. `canAdd` is derived from the same state, so it is recalculated on every keystroke. `disabled` stops presses; the extra style makes that visible. `Date.now()` gives a unique id so keys stay unique even if two habits share a label.

Common mistakes

- Clearing with `label = ""`. Reassigning the const fails, and a plain variable wouldn't re-render anyway. Use the setter.
- Checking `label !== ""` instead of `label.trim()`. A label of three spaces would sneak through.
- Using the label as the key. Two habits with the same label would collide and trigger the duplicate-key warning.

Café: the 0 && trap

✓ Solution · App.tsx

```tsx
import { useState } from "react";
import { View, Text, Pressable, ScrollView, StyleSheet } from "react-native";

type MenuItem = { name: string; popular: boolean };

const menuItems: MenuItem[] = [
  { name: "Latte", popular: true },
  { name: "Chai", popular: false },
  { name: "Mocha", popular: true },
];

export default function App() {
  const [menu, setMenu] = useState(menuItems);
  const popularCount = menu.filter((item) => item.popular).length;

  const clearPopular = () => {
    setMenu(menu.map((item) => ({ ...item, popular: false })));
  };

  return (
    <View style={styles.container}>
      <Text style={styles.title}>Café menu</Text>
      {/* A real boolean on the left, and a ternary for the "else" message */}
      {popularCount > 0 ? (
        <View style={styles.banner}>
          <Text style={styles.bannerText}>
            {popularCount} popular {popularCount === 1 ? "item" : "items"}
          </Text>
        </View>
      ) : (
        <Text style={styles.muted}>No popular items yet</Text>
      )}
      <ScrollView style={styles.list}>
        {menu.map((item) => (
          <View key={item.name} style={styles.row}>
            <Text style={styles.name}>{item.name}</Text>
            {/* Safe: item.popular is a boolean, and false renders nothing */}
            {item.popular && <Text style={styles.badge}>Popular</Text>}
          </View>
        ))}
      </ScrollView>
      <Pressable style={styles.button} onPress={clearPopular}>
        <Text style={styles.buttonText}>Clear popular flags</Text>
      </Pressable>
    </View>
  );
}

// const styles = StyleSheet.create({ ... }) is unchanged
// from the starter file on the Practice page.
```

Why this works. `popularCount && <Banner />` evaluates to `popularCount` when it is 0, so React tries to render the number `0` directly inside a `View`. On the web that would just print "0"; in React Native it is a string outside `<Text>`, which is an error. The rule: **never put a number on the left of `&&`**. Use `count > 0 &&`, or a ternary when you want an "else".

Common mistakes

- "Fixing" it with `{popularCount.toString() && ...}`. The string "0" is truthy, so the banner shows "0 popular items".
- Worrying about `item.popular && ...`. That one is fine; `false` renders nothing.
- Wrapping the whole banner in `<Text>` to silence the error. The banner would then show a stray 0.

JS first, types second: type a working app

✓ Solution · App.tsx

```
import { useState } from "react";
import { View, Text, TextInput, Pressable, ScrollView, StyleSheet } from "react-native";

// 1. The data shape: write this first
type Plant = { name: string; waterEveryDays: number; thirsty: boolean };

// 2. Props are one object; describe it and annotate the destructured parameter
type PlantCardProps = { plant: Plant };

const PlantCard = ({ plant }: PlantCardProps) => (
  <View style={styles.card}>
    <Text style={styles.name}>{plant.name}</Text>
    <Text style={styles.meta}>Every {plant.waterEveryDays} days</Text>
  </View>
);

// 3. A callback prop has a function type
type AddPlantFormProps = { onAdd: (plant: Plant) => void };

const AddPlantForm = ({ onAdd }: AddPlantFormProps) => {
  const [name, setName] = useState(""); // inferred string: no annotation needed
  const [days, setDays] = useState("7"); // TextInput works with strings
  return (
    <View style={styles.form}>
      <TextInput style={styles.input} value={name} onChangeText={setName} placeholder="Plant
   name" />
      <TextInput style={styles.input} value={days} onChangeText={setDays} keyboardType="number-
   pad" placeholder="Water every N days" />
      <Pressable
        style={styles.button}
        // TS now checks this object against Plant: every field must be present
        onPress={() => onAdd({ name: name, waterEveryDays: Number(days) || 7, thirsty: true })}
      >
        <Text style={styles.buttonText}>Add plant</Text>
      </Pressable>
    </View>
  );
};

export default function App() {
  // 4. Empty initial value: give the type explicitly (otherwise never[])
  const [plants, setPlants] = useState<Plant[]>([]);

  // 5. A standalone function's parameter needs a type
  const addPlant = (plant: Plant) => {
    setPlants([...plants, plant]);
  };

  return (
    <View style={styles.container}>
      <Text style={styles.title}>Plant care</Text>
      <ScrollView style={styles.list}>
        {/* (plant) here is inferred as Plant from plants: Plant[] */}
        {plants.map((plant) => (
          <PlantCard key={plant.name} plant={plant} />
        ))}
      </ScrollView>
      <AddPlantForm onAdd={addPlant} />
    </View>
  );
}

// const styles = StyleSheet.create({ ... }) is unchanged
// from the starter file on the Practice page.
```

Why this works. TypeScript can infer most types from values: `useState("")` is a string, and `useState(startingPlants)` would be `Plant[]`. It **can't** infer from nothing, which is why an empty `useState([])` and unannotated standalone parameters need help. Callbacks written inline, like `(plant) =>` in `map` or `(text) =>` in `onChangeText`, get their types from where they are used, so you can leave them alone. That is the "JS first, types second" workflow: get it working, then add types where TypeScript asks.

Common mistakes

- Typing every variable by hand, like `const [name, setName] = useState<string>("")`. It's not wrong, just slower; inference already knows.
- Typing the props as `({ plant: Plant })`. Inside a destructuring pattern, `plant: Plant` means *rename* `plant` to a variable called `Plant`. The type goes after the braces: `({ plant }: PlantCardProps)`.
- Silencing errors with `any`. It removes the check that would have caught a missing field.

Café: header, scrolling list, pinned footer

✓ Solution · App.tsx

```
import { View, Text, Pressable, ScrollView, StyleSheet } from "react-native";

type MenuItem = { name: string; popular: boolean };

const menu: MenuItem[] = [
  { name: "Latte", popular: true },
  { name: "Chai", popular: false },
  { name: "Mocha", popular: true },
  { name: "Matcha", popular: false },
];

export default function App() {
  return (
    <View style={styles.container}>
      <Text style={styles.title}>Café menu</Text>
      <ScrollView style={styles.list}>
        {menu.map((item) => (
          <View key={item.name} style={styles.row}>
            <Text style={styles.name}>{item.name}</Text>
            {item.popular && <Text style={styles.badge}>Popular</Text>}
          </View>
        ))}
      </ScrollView>
      <View style={styles.footer}>
        <Text style={styles.footerText}>{menu.length} items</Text>
        <Pressable style={styles.button}>
          <Text style={styles.buttonText}>Order</Text>
        </Pressable>
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  // flex: 1 on the root = fill the screen. Children stack in a column by default.
  container: { flex: 1, paddingTop: 60, paddingHorizontal: 20, paddingBottom: 30,
   backgroundColor: "#f6f7fc" },
  title: { fontSize: 28, fontWeight: "800", marginBottom: 12 },
  // The list grows to fill whatever the title and footer don't use
  list: { flex: 1 },
  // Row: main axis is horizontal, so justifyContent spreads left/right,
  // and alignItems centers on the cross (vertical) axis
  row: {
    flexDirection: "row",
    justifyContent: "space-between",
```

```
    alignItems: "center",
    backgroundColor: "#ffffff",
    borderRadius: 12,
    padding: 14,
    marginBottom: 8,
  },
  name: { fontSize: 18, fontWeight: "600" },
  badge: { backgroundColor: "#f39c12", color: "#ffffff", fontWeight: "800", fontSize: 13,
   paddingHorizontal: 10, paddingVertical: 4, borderRadius: 999, overflow: "hidden" },
  footer: {
    flexDirection: "row",
    justifyContent: "space-between",
    alignItems: "center",
    paddingTop: 12,
    borderTopWidth: 1,
    borderTopColor: "#e3e6f3",
  },
  footerText: { fontSize: 17, fontWeight: "700" },
  button: { backgroundColor: "#7cb342", borderRadius: 12, paddingHorizontal: 22, paddingVertical:
   12 },
  buttonText: { color: "#ffffff", fontSize: 17, fontWeight: "700" },
});
```

Why this works. Think in **axes**. `justifyContent` works along the main axis (the `flexDirection`), and `alignItems` works across it. Rows flip the main axis to horizontal, so `space-between` pushes the name and badge to opposite ends, and `alignItems: "center"` lines them up vertically. The footer stays at the bottom without any absolute positioning, because the list's `flex: 1` soaks up all the space between the header and the footer.

Common mistakes

- Forgetting `flex: 1` on the container. The list then has no bounded height, so it can't scroll properly and the footer may not sit at the bottom.
- Expecting `justifyContent: "center"` to center vertically inside a row. In a row, vertical is the cross axis, which is `alignItems`.
- Using `<Button style={...}>`. `Button` has no `style` prop; use a `Pressable` with a `Text` child for custom buttons.

Writing sprint: a countdown timer with useEffect and cleanup

✓ Solution · App.tsx

```
import { useState, useEffect } from "react";
import { View, Text, Pressable, StyleSheet } from "react-native";

const SPRINT_SECONDS = 60;

export default function App() {
  const [secondsLeft, setSecondsLeft] = useState(SPRINT_SECONDS);
  const [running, setRunning] = useState(false);

  // Runs after the first render and whenever `running` changes
  useEffect(() => {
    if (!running) return; // paused: no interval, nothing to clean up

    const id = setInterval(() => {
      // Updater form: always uses the latest value, not a stale snapshot
      setSecondsLeft((s) => (s > 0 ? s - 1 : 0));
    }, 1000);

    // Cleanup: runs before the effect re-runs (running changed) and on unmount
    return () => clearInterval(id);
  }, [running]);

  // Stop automatically at zero
```

```
  useEffect(() => {
    if (secondsLeft === 0) setRunning(false);
  }, [secondsLeft]);

  const reset = () => {
    setRunning(false);
    setSecondsLeft(SPRINT_SECONDS);
  };

  const minutes = Math.floor(secondsLeft / 60);
  const seconds = String(secondsLeft % 60).padStart(2, "0");

  return (
    <View style={styles.container}>
      <Text style={styles.title}>Writing sprint</Text>
      <Text style={styles.big}>
        {minutes}:{seconds}
      </Text>
      <Text style={styles.sub}>{secondsLeft === 0 ? "Time! Pens down." : "Keep going!"}</Text>
      <View style={styles.row}>
        <Pressable
          style={styles.button}
          // Don't start a finished sprint; Reset first
          onPress={() => setRunning(!running && secondsLeft > 0)}
        >
          <Text style={styles.buttonText}>{running ? "Pause" : "Start"}</Text>
        </Pressable>
        <Pressable style={styles.ghost} onPress={reset}>
          <Text style={styles.ghostText}>Reset</Text>
        </Pressable>
      </View>
    </View>
  );
}

// const styles = StyleSheet.create({ ... }) is unchanged
// from the starter file on the Practice page.
```

Why this works. The interval is an **external system**: React doesn't manage it, so you start it in an effect and stop it in the cleanup. With `[running]`, pressing Pause changes `running`, React runs the cleanup (clearing the old interval), and the effect re-runs and does nothing. The updater form matters because the interval callback was created during one render; with `setSecondsLeft(secondsLeft - 1)` it would keep using that render's snapshot and get stuck at 59.

Common mistakes

- Calling `setInterval` directly in the component body. It starts a new interval on **every** render, and they pile up.
- Forgetting the cleanup. Each Start adds another interval, so the countdown speeds up.
- Leaving out the dependency array: `useEffect(fn)` runs after every render, so the interval would be torn down and restarted every second.

Build A: complete final App.tsx

One file, runnable in Snack, fully typed, with StyleSheet.

✓ Final · App.tsx

```
import { useState } from "react";
import { View, Text, TextInput, Pressable, ScrollView, Switch, StyleSheet } from "react-native";

// ---------- Types ----------
type MenuItem = {
  name: string;
  popular: boolean;
};
```

```
type ItemCardProps = {
  item: MenuItem;
  onTogglePopular: (name: string) => void;
  onRemove: (name: string) => void;
};

type AddItemFormProps = { onAdd: (item: MenuItem) => void };

// ---------- Data ----------
const menuItems: MenuItem[] = [
  { name: "Latte", popular: true },
  { name: "Chai", popular: false },
  { name: "Banana bread", popular: true },
];

// ---------- Child: one card ----------
const ItemCard = ({ item, onTogglePopular, onRemove }: ItemCardProps) => (
  <View style={[styles.card, item.popular && styles.cardPopular]}>
    <View style={styles.cardTop}>
      <Text style={styles.cardName}>{item.name}</Text>
      {item.popular && (
        <View style={styles.badge}>
          <Text style={styles.badgeText}>Popular</Text>
        </View>
      )}
    </View>
    <View style={styles.cardActions}>
      {/* Arrow functions: run only on press, with this card's name */}
      <Pressable style={styles.smallButton} onPress={() => onTogglePopular(item.name)}>
        <Text style={styles.smallButtonText}>{item.popular ? "Unmark" : "Mark popular"}</Text>
      </Pressable>
      <Pressable style={styles.smallButton} onPress={() => onRemove(item.name)}>
        <Text style={styles.smallButtonText}>Remove</Text>
      </Pressable>
    </View>
  </View>
);

// ---------- Child: the add form ----------
const AddItemForm = ({ onAdd }: AddItemFormProps) => {
  // Local state: only the form needs the in-progress text
  const [name, setName] = useState("");
  const [popular, setPopular] = useState(false);
  const canAdd = name.trim() !== "";

  const handleAdd = () => {
    // Build the object the MenuItem type expects, then send it UP
    onAdd({ name: name.trim(), popular });
    setName("");
    setPopular(false);
  };

  return (
    <View style={styles.form}>
      <TextInput style={styles.input} value={name} onChangeText={setName} placeholder="Item name"
   />
      <View style={styles.switchRow}>
        <Text style={styles.switchLabel}>Popular?</Text>
        <Switch value={popular} onValueChange={setPopular} />
      </View>
      <Pressable
        style={[styles.addButton, !canAdd && styles.addButtonDisabled]}
        disabled={!canAdd}
        onPress={handleAdd}
      >
        <Text style={styles.addButtonText}>Add item</Text>
      </Pressable>
```

```
    </View>
  );
};

// ---------- Parent: owns the state ----------
export default function App() {
  const [menu, setMenu] = useState(menuItems); // inferred: MenuItem[]

  const addMenuItem = (item: MenuItem) => {
    // Names are used as keys, so keep them unique
    if (menu.some((m) => m.name === item.name)) return;
    setMenu([...menu, item]); // new array, so React re-renders
  };

  const togglePopular = (name: string) => {
    setMenu(menu.map((m) => (m.name === name ? { ...m, popular: !m.popular } : m)));
  };

  const removeItem = (name: string) => {
    setMenu(menu.filter((m) => m.name !== name));
  };

  // Derived values: never stored in state
  const popularCount = menu.filter((m) => m.popular).length;

  const cards = menu.map((item) => (
    <ItemCard
      key={item.name}
      item={item}
      onTogglePopular={togglePopular}
      onRemove={removeItem}
    />
  ));

  return (
    <View style={styles.container}>
      <Text style={styles.title}>Café Menu</Text>
      <Text style={styles.count}>
        {menu.length} items · {popularCount} popular
      </Text>
      {menu.length === 0 ? (
        <Text style={styles.empty}>The menu is empty. Add your first item below.</Text>
      ) : (
        <ScrollView style={styles.list}>{cards}</ScrollView>
      )}
      <AddItemForm onAdd={addMenuItem} />
    </View>
  );
}

// ---------- Styles ----------
const styles = StyleSheet.create({
  container: { flex: 1, paddingTop: 60, paddingHorizontal: 20, paddingBottom: 30,
   backgroundColor: "#f6f7fc" },
  title: { fontSize: 30, fontWeight: "800", color: "#1b1f3b" },
  count: { fontSize: 16, color: "#5f6690", marginBottom: 12 },
  list: { flex: 1 },
  empty: { flex: 1, fontSize: 17, color: "#5f6690", textAlign: "center", marginTop: 40 },
  card: { backgroundColor: "#ffffff", borderRadius: 14, padding: 14, marginBottom: 10 },
  cardPopular: { borderLeftWidth: 6, borderLeftColor: "#f39c12" },
  cardTop: { flexDirection: "row", justifyContent: "space-between", alignItems: "center" },
  cardName: { fontSize: 18, fontWeight: "700", flex: 1 },
  badge: { backgroundColor: "#f39c12", borderRadius: 999, paddingHorizontal: 10, paddingVertical:
   4 },
  badgeText: { color: "#ffffff", fontWeight: "800", fontSize: 13 },
  cardActions: { flexDirection: "row", gap: 8, marginTop: 10 },
```

```
  smallButton: { borderWidth: 1, borderColor: "#d5d9ec", borderRadius: 8, paddingHorizontal: 10,
   paddingVertical: 6 },
  smallButtonText: { fontSize: 14, fontWeight: "600", color: "#3c4270" },
  form: { gap: 10, paddingTop: 12, borderTopWidth: 1, borderTopColor: "#e3e6f3" },
  input: { backgroundColor: "#ffffff", borderWidth: 1, borderColor: "#d5d9ec", borderRadius: 10,
   padding: 12, fontSize: 17 },
  switchRow: { flexDirection: "row", justifyContent: "space-between", alignItems: "center" },
  switchLabel: { fontSize: 17, fontWeight: "600" },
  addButton: { backgroundColor: "#6c5ce7", borderRadius: 12, paddingVertical: 14, alignItems:
   "center" },
  addButtonDisabled: { backgroundColor: "#c9cde0" },
  addButtonText: { color: "#ffffff", fontSize: 17, fontWeight: "700" },
});
```